Department of electrical and electronic engineering

Event-driven multi-class detection using neuromorphic vision sensor in traffic scenes based on YOLO detector

Submitted by

Author name: **Huang Haiyang**

Author ID: 3036379641

M.Sc. (Eng) (EEE)

Dissertation supervisor: Dr. Edmund Lam

A dissertation submitted to The University of Hong Kong

for the degree of Master of Engineering

Date of submission: Aug 1, 2025

Declaration

I am Huang Haiyang, and I make a formal declaration hereby that this dissertation is done by my own without plagiarizing or copying from published papers, submitted reports, and dissertations from any universities or any institutions for a degree or qualification.

signed

Acknowledgement

First of all, I would like to express my sincere gratitude to my supervisor Prof. Edmund Lam. During the design period, he gave me a high degree of freedom to design the system with my brain's thinking. As the first assessor of my first and second benches. He also provided many valuable suggestions to me, highlighting the importance of low latency in the applications of real-time monitoring and taught me trying to reduce the complexity of preprocessing steps to increase FPS performance.

Secondly, I also express my appreciation to my second assessor Dr. Vincent Tam, who told me that the evaluation process of the trained model is important to show in the dissertation during the midterm and final presentations.

Last but not least, thanks to my parents and family members, who support the funds to allow me to stand on their shoulders to broaden my horizons and enjoy life in HKU.

Contents

List of Figures

List of Tables

Abbreviations

CLS: CMOS Image Sensor
CNN: Convolutional Neural Network
SVM: Support Vector Machine
HOG: Histogram of Oriented Gradient
DVS: Dynamic Vision Sensor
SNNs: Spiking Neural Networks
SAE: Surface of Active Events
LIF: Leaky Integrate-and-fire
mAP: Mean Average Precision
VIA: VGG Image Annotator
BRAT: Brat Rapid Annotation Tool
SPPF: Spatial Pyramid Pooling Fast
PANet: Path Aggregation Network
MBGD: Mini Batch Gradient Descent
BGD: Batch Gradient Descent
NMS: Non-Maximum Suppression
IoU: Intersection-over-Union
AP: Average Precision

Abstract

Amidst the ongoing evolution of edge computing devices and AI detection models, research on autonomous driving and safeguard driving assistance has emerged nowadays. This paper contributes to designing a CNN-based traffic monitoring system based on simulated event data stream by using Python programming language, which is capable of detecting the instances of cars, pedestrians, motorcyclists and cyclists in real-time traffic scenes under any extreme weathers. All versions of YOLOv5 and YOLOv8 are trained to evaluate their performance, explore the effects of varying batch size and epochs during the training process. The results show that training models under 16 batch sizes with 125 epochs have the best evaluation performances without the trend of over-fitting. Then inferred the YOLO detectors with simulated event camera and preprocessing of frequency-based encoding and blending, finally finding out that both n-versions of the YOLO model are more suitable to carry out the real-time monitoring task with more than 70FPS under a GPU environment with the average precision of more than 80%. The counting function is also developed to record the total target numbers within each video frame, which can integrate with the speaker to alarm drivers when the road ahead is congested by targets.

Keywords: event camera, event stream, real-time traffic monitoring, YOLO detector, counting function

Chapter 1: Introduction

1.1 Background

The traffic monitoring system is a core component in maintaining the traffic participants' driving safety and making an effort to mitigate traffic jams in real-time based on machine learning algorithms [1]. It has the ability to utilize sensors and cameras to detect, classify and interpret traffic conditions in a certain area's vicinity. In recent years, with the development of edge computing devices and technological advancements, the AI-based object detection system for traffic monitoring and surveillance has become very popular. Edge devices with computing capabilities can connect cameras, sensors and integrate machine learning algorithms to realize the function of detecting and counting traffic tools based on the delivered object-detection model, which enables traffic lights to intelligently adjust the timing of signal changes based on real-time traffic flow [2]. Nowadays, the transition from gasoline vehicles to electric ones has enabled more sensors to be integrated into the car's autonomous driving and intelligent driving systems. With the support of object detection technology, the smart driving system can activate the brakes and issue a warning sound to alert the driver when the monitoring system detects the presence of vehicles or pedestrians within a certain detection range. What is more, for an autonomous driving system, it is a key block to activate the emergency measures without human intervention to ensure the passengers' safety. Thanks to the AI-based road monitoring system, drivers can be informed in advance whether the road is congested, and therefore, consider changing the current driving route or reducing the vehicle speed [3].

Over the last twenty years, many researchers have succeeded in using the conventional CMOS image sensor (CLS) to monitor and capture images of vehicles continuously in a day to analyze the traffic behavior and classify the captured vehicles, lanes and pedestrians automatically in real-time. However, the utilize of conventional image sensors to do the real-time object detection tasks has drawbacks. The first point is that it generally captures the entire scene at a predetermined frame rate and outputs every static frame with fixed intervals, regardless of any target activity in the scene, which constrains the response speed of camera, and a large amount of captured frames are redundant or background information rather than the foreground [4]. The second point is that it captures the absolute luminance of pixels for all frames instead of capturing the difference between the adjacent frames, which is not optimize for traffic monitoring. Low frame rates have the potential risk of missing key visual information, while high

frame rates produce large amounts of redundant data to make it a heavier burden to consume extra computational resources. It becomes problematic in applications that require high data throughput and low latency. Recently, the neuromorphic camera is a new product as a better choice to be selected for carrying out the simultaneous object detection work, which is based on the principle of the biological retina. A neuromorphic camera is a kind of image sensor which only responds to changes in brightness compared with the current levels. It outputs an asynchronous stream of events triggered by changes in scene illumination [5]. Therefore, compared with conventional CLS, it can detect much more target data at the event stream level with low latency instead of inputting each frame at the frame level, allowing it to handle a wider range of lighting conditions, lower power consumption, and higher operation efficiency.

In conclusion, even though lots of research achievements have been made in the field of traffic detection, the application and research of the AI-based neuromorphic image analysis based on the traffic object detection model still require supplementation and further exploration. In this paper, an event-based smart monitoring traffic system is proposed by constructing a custom CNN-based YOLO detector to detect and classify vehicles and pedestrians in real-time.

1.2 Literature review

As more advanced object detection algorithms are explored, traffic multi-class target detection is a hot topic to be debated as a solution to reduce the accident rate, relieve traffic congestion and bring the innovation of autonomous driving. There are two main classes of machine learning algorithms used to realize the object detection tasks, which are the traditional artificial feature extraction algorithm and the deep-learning algorithm, respectively [6]. Before deep learning, object detectors were built as multi-stage pipelines using hand-crafted features and conventional classifiers. In [7], the author used the histogram of oriented gradient (HOG) features to describe the pedestrians' appearance manually before training the support vector machine (SVM) algorithm to do the pedestrian detection work.

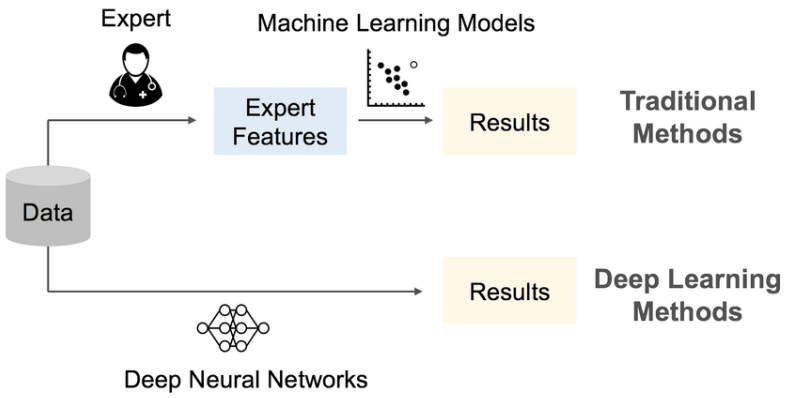


Figure 1: Comparison between traditional method and deep learning method

As shown in Fig.1, the feature extraction requires the researcher to gain a deep understanding of what the target object is to be detected. Even though detection results from the traditional algorithm are acceptable in the surveillance system, it still struggles with highly cluttered scenarios and variations in object appearance [8]. In contrast, the deep learning algorithm is a subset of machine learning that mimics the functioning of the human brain through neural networks. Unlike its traditional counterpart, deep learning autonomously learns from input data instead of relying on handcrafted features. The convolutional neural network (CNN) is the most representative model in the deep learning field, which is made up of neurons to replicate human cognitive capabilities.

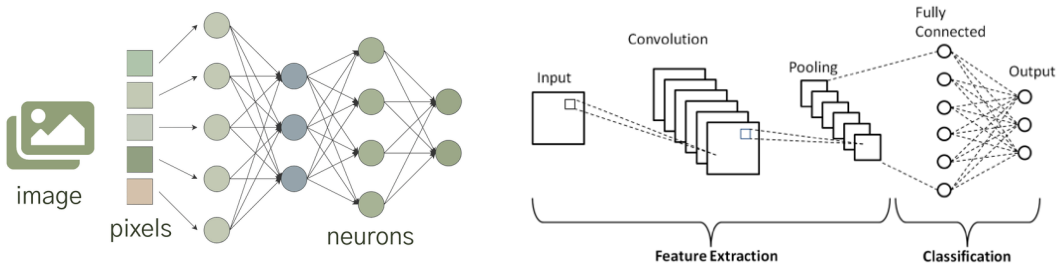


Figure 2: Neural Mechanism of CNNs (Right)

Figure 3: Basic Structure of CNNs(Left)

As is demonstrated in Fig.2 and Fig.3, each neuron has a learnable weight and bias that can automatically learn to identify high-level features and extract local feature maps by processing each input frame through convolutional layers and hidden layers that detect patterns and classify identities [9]. The function of the pooling layer is to downsample the extracted feature maps in order to reduce the computational cost and number of parameters. Finally, the fully connected layers produce class scores for the detected objects relative to the ground truth labels of training [10]. R-CNN family and YOLO family are two prevalent CNN-based object detection algorithms. R-CNN initially belongs to a two-stage detector that is known as higher detection accuracy

compared with the YOLO detector, which is suitable for detecting fine details of objects without the requirement of time cost [11]. R-CNN is more resource-intensive and has lower inference speed than the YOLO family; its characteristics mean that it is not suitable for performing the real-time monitoring task. In contrast, YOLO is a single-shot detector, which predicts bounding boxes and class probabilities for objects in a single pass through the neural network. This makes it more efficient and faster for detection compared to R-CNN [12]. In [13], the researcher trained 500 images of human with different backgrounds using both R-CNN and YOLO model, the results showed that R-CNN had a higher accuracy when carrying the asynchronous human detection work, the accuracy is highly dependent on inference time, it cannot detect all human figures correctly at a tiny latency. However, there are lots of successful real-time traffic detection cases with various versions of the YOLO detector, even though sacrificing a little accuracy compared with R-CNN. In [14], the author trained a YOLOv3 model to detect and classify the main traffic participants, including cars, trucks, pedestrians, traffic signs and traffic lights. Testing the unknown 300 images to get the precision of 63% and the recall rate of 55%. The testing video reached 25 FPS, which is acceptable for the goal of real-time monitoring. The author in [16] collected 4000 images of traffic signs from videos and labelled each image to build a dataset for training and testing, training the YOLOv3 and YOLOv4 models, respectively. The validation process indicated that the accuracy of detection reached at highest with 95.09% based on YOLOv4, which is 2.83% higher than the older version of v3.

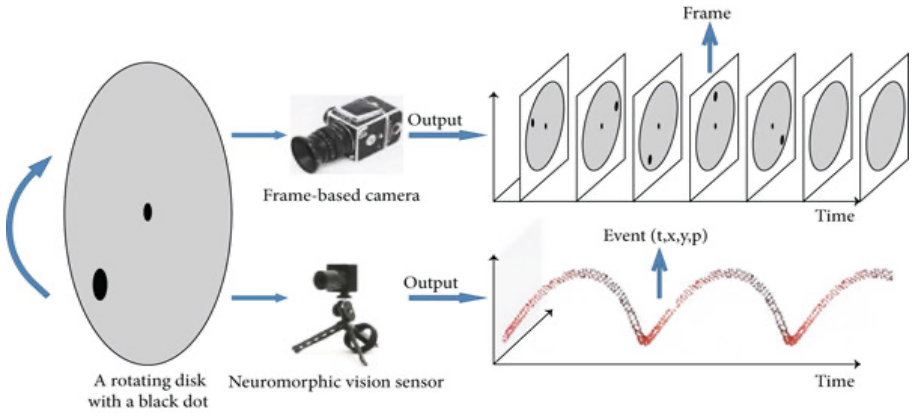


Figure 4: Comparison of the output of frame-based and event-based sensor

The component of the equipped sensor is an important hardware to influences the model detection results. Fig. 4 demonstrates that the traditional frame-based camera captures frames at a fixed rate, no matter whether the position of the black dot is changed or the disk rotates, causing the redundant and repeated dataset collection to consume more computational resources and lower frames per second. The extreme weather

conditions and rapid motion can reduce the visibility of scenes and lead the motion blur, which drops the detection accuracy and increases the probability of false classification [15]. However, the neuromorphic camera can capture the key information of the difference between two adjacent frames since it only outputs the event stream when a change in intensity beyond a pre-set threshold, which greatly improves the efficiency, saves memory to store unrelated frames and reduces complexity [16].

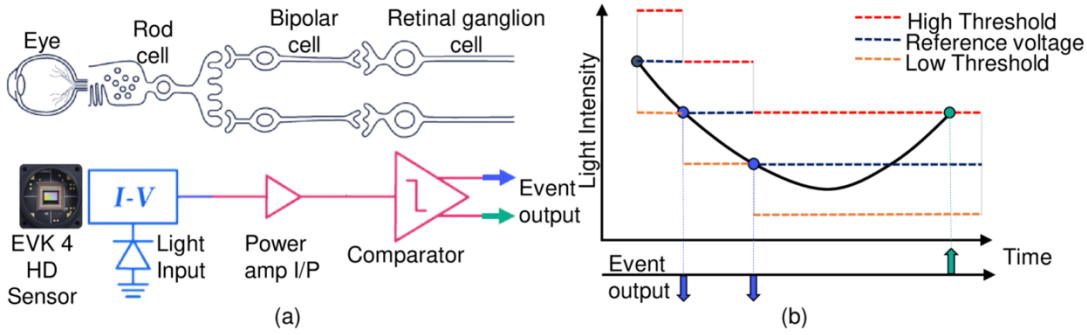


Figure 5: Working Mechanism of the Neuromorphic Camera

Fig.5 shows the working mechanism of the Neuromorphic camera that is inspired by the human retina, where the eye, rod cell, bipolar cell and retinal ganglion cell extract visual signals without redundant information. In Fig.5(a), the eye captures incoming photons and produces a proportional current, which is referred to as the camera's pixel. Then the I-V converter is used to transform the photocurrent into a voltage signal. For the next step, the power amplifier buffers and amplifies the voltage to ensure the electrical signal is large enough to drive the next stage [15]. In Fig.5(b), the comparator is then used to compare the incoming instantaneous voltage between two thresholds; the neuromorphic camera starts to record the input frame if the magnitude of the reference voltage is above the high threshold. In contrast, there is no event stream data to be output when the electric signal is below the high threshold [17].

The Neuromorphic camera is also referred to as dynamic vision sensor. Currently, more and more in-depth traffic detection research is being conducted based on the dynamic vision sensor (DVS). Wan [18] in 2021 implemented a CNN-based YOLOv3 detection model to be integrated with the DVS to detect pedestrians in real-world multiple scenes. Its experiment results found that incorporating DVS into the system gain 20% improvement in detection speed compared to conventional image sensors. Unlike traditional frame-based sensors, the DVS significantly reduces the computational resources needed to analyze motion across video frames. Chen [19] in 2018 trained a CNN-based Yolo tiny version detector by using pseudo-labels to detect cars from DVS

data, which successfully implements object detection at 100 frames per second under ego-motion in a real environment, achieving an average precision of 40.3% when evaluated against annotated ground truth. Additionally, the designed model can also detect in conditions of motion blur and low light, despite the absence of such scenarios in the pseudo-label generation process. Dewangan [20] in 2021 proposed an intelligent vehicle system that employed a DVS for real-time lane detection based on the Yolo detector. The system utilized a Raspberry Pi and a camera module to capture road images, then processed them to identify lane boundaries and guide vehicle movement. By analyzing lane geometry and driving behavior, the system supports autonomous lane following and enhances driving safety under controlled conditions. Chen [21] in 2023 explored the use of spiking neural networks (SNNs) and convolutional neural networks (CNNs) with simulated DVS data for autonomous driving tasks, focusing on detecting traffic light changes in photo-realistic simulations. The study showed that event-based data capturing only brightness changes improved detection accuracy compared to traditional RGB frame-level input. Although SNNs offered advantages in biological plausibility and energy efficiency, CNNs trained on DVS data still outperformed SNNs in accuracy. The research highlights the potential of event-based sensing for energy-efficient, real-time perception in dynamic driving environments.

Based on the research of various applications of real-time traffic participants' detection systems, there is a growing trend to transition from frame-based sensors to event-based sensors with the CNN-based object detection model. Even though the different versions of the YOLO detection model and event-based sensors have been integrated together to get high precision, more detection classes and assisted functions are required to complement them for improving driving safety.

1.3 Objectives and paper organization

This paper aims to develop an event-based smart traffic monitoring system which can detect cars, pedestrians, cyclists and motorcyclists based on YOLOv5 and YOLOv8 detectors by considering the least cost of computation and latency. Complete evaluation and testing processes are also experimentally finished and analyzed in this paper. Additionally, the designed system also realize the function of counting the target vehicles in one frame. The contributions of this paper can be summarized in the list below:

1) The dynamic vision sensor is simulated with the blended output to augment the foreground moving objects without much loss of FPS to increase the detection accuracy.

2) The CNN-based YOLOv5 and YOLOv8 detection models are trained with the custom-collected frames of videos of traffic participants with the step of manual annotation of three-class datasets of cars, pedestrians and riders. The performances of these models of different versions are evaluated and tested with robust methods.
3) The methods of multi-threads, non-maximum suppression and letterbox process are proposed and applied to speed up and optimize the inference of the detection model based on CPU and GPU, respectively.

Application scenario: Real-time traffic monitoring sensor integrated in vehicles, edge-computing devices for monitoring road conditions asynchronously,

Chapter 2 Problem Statement

There are some practical challenging issues that need to be faced and solved to realize the system design. In this chapter, the main difficulties will be described with a brief statement. The proposed solutions to each issue will be introduced and explained in the next chapter.

2.1 The simulation of DVS

In this paper, no practical hardware is deployed to implement the designed system due to budget constraints. The neuromorphic sensor is required to be simulated by Python code on a Windows-based PC. It can be realized by processing the captured RGB frame from the conventional camera into a grayscale output of event streams, highlighting the moving objects by setting a threshold of light intensity.

2.2 Preparation work before YOLO detector training

2.2.1 The problem with the output from DVS cannot be directly regarded as the input of the YOLO framework

The simulated DVS generates the event (x_n, y_n, t_n, p_n) at the timestamp of t_n with the polarity of [-1, +1] rather than a frame-based form when the brightness level changes above a threshold. However, its output cannot be used as the input to train with the YOLO framework which only directly supports the frame-level input [22]. Therefore, the process of converting event stream data to frame-level image is necessary to train a YOLO detection model, where each frame can be regarded as one image. The method of event-frame encoding based on frequency is proposed to slice event-based data into 2-D images before feeding them into artificial neurons for automatic learning. The method of blending is also used to improve the contrast level of the moving foreground.

2.2.2 The problem with collection of the DVS dataset

In this paper, a custom-collected dataset is created by collecting the key frames in videos that include cars, pedestrians, cyclists and motor cyclists. Then, the annotation process is required to label the foreground with the ground truth bounding boxes in each image. After that, the dataset needs to be split into three parts for training, which are training images, validation images and testing videos. As shown in Fig.6, validation mean average precision (mAP) measured on the blue set is a reliable approximation result when the training set and validation set are in the traffic scene under DVS with the same patterns. The validation result can be lower than the expectation if the validation set contains the unlearned features and different backgrounds [23]. For instance, poor generalization results will be caused if the frames captured from a quiet suburban street are used for training while those from a busy urban intersection are prepared for validation. The reason is that convolutional layers cannot recognize unknown features or non-existent features from training set directly in the validation process. What is more, the cheating invalid mAP will be generated if the part images in the validation set overlap with the training set, the result of recall rate will be higher than the real performance. Therefore, organizing the collected images and annotating labels to construct a perfect dataset for YOLO training is also a challenge. The detection accuracy is highly dependent on the resolution of images and the quality of annotation with bounding boxes.

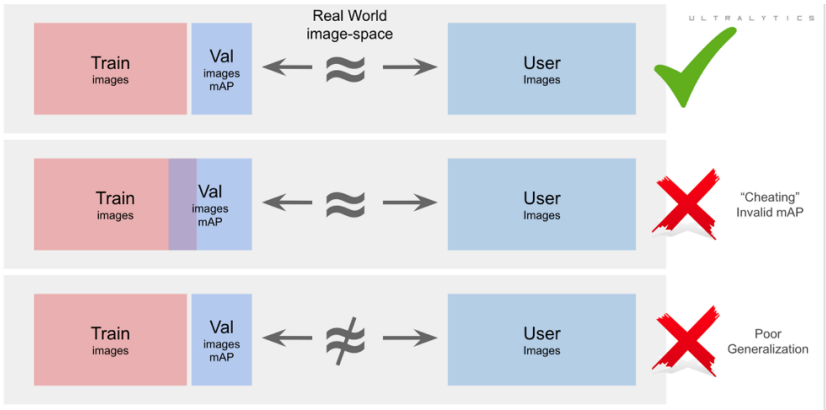


Figure 6: The common pitfalls of YOLO dataset composition

2.2.3 The selection of model version and parameters

As shown in Fig.7, since the invention of YOLOv1 to be used for regression tasks with object detection in 2015, the related applications have boomed in conduct. After undergoing multiple iterations to improve the model performance, the latest version comes to 11 in Sept of 2024, which means that YOLO detection model is so popular for engineers to deploy for real-time detection works with the tolerance of less accuracy

than two-stage detectors like R-CNN. This paper is dedicated to deploying the model with the lowest cost and time latency. In [24], the author deployed the YOLO detection model to monitor the behaviors of traffic signs with the model versions from v3 to v12. The performances of each version are analyzed to show that the detection accuracy and recall rate of YOLOv8 and YOLOv5 rank first place and second place, respectively. However, the detailed parameters like epoch, batch size, learning rate, drop-out rate and data split for training are required to be fine-tuned based on the model validation results, which are challenging to obtain acceptable results at the first training time. In chapter 3, each training parameter and the different sub-versions of YOLOv8 and YOLOv5 will be introduced and analyzed. The method of non-maximum suppression is also applied to filter the inferred overlapped bounding boxes for each targeted object. In chapter 4, the validation results of these models will be compared and make a wise decision to find out the optimal model based on the consideration of cost and the requirements of inference speed. In order to realize the higher inference speed of YOLO model, the method of multi-threads is also proposed to be applied for improving the FPS in the environment of CPU and GPU, respectively.

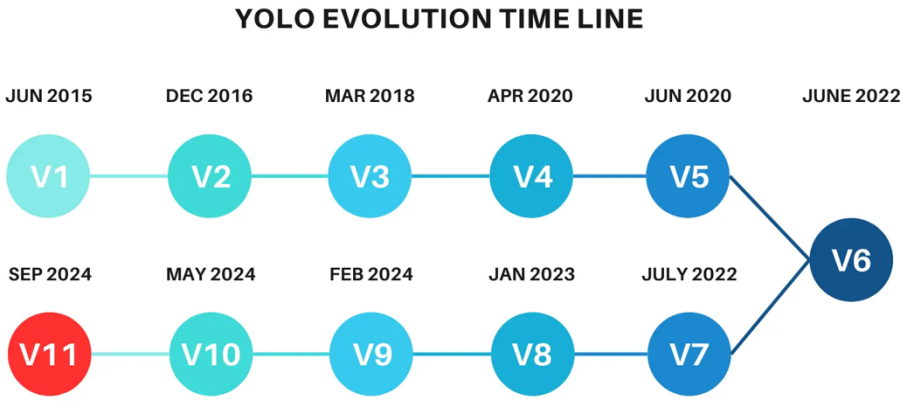


Figure 7: The timeline for the evolution of YOLO model

2.2.4 Activate function of counting bounding boxes in current frame

The difficulty in this field is that the accuracy of counting results is highly dependent on the trained detection model. And how to apply the counting function in real-time with the inferred model also confuses to deployment. The proposed design also mentions that the speaker will output a voice notice to remind the vehicle driver, and the integration of these two functions to simultaneously activate during the detection process is also a challenge for coding.

The draft flow chart drawing of the proposed design operation:

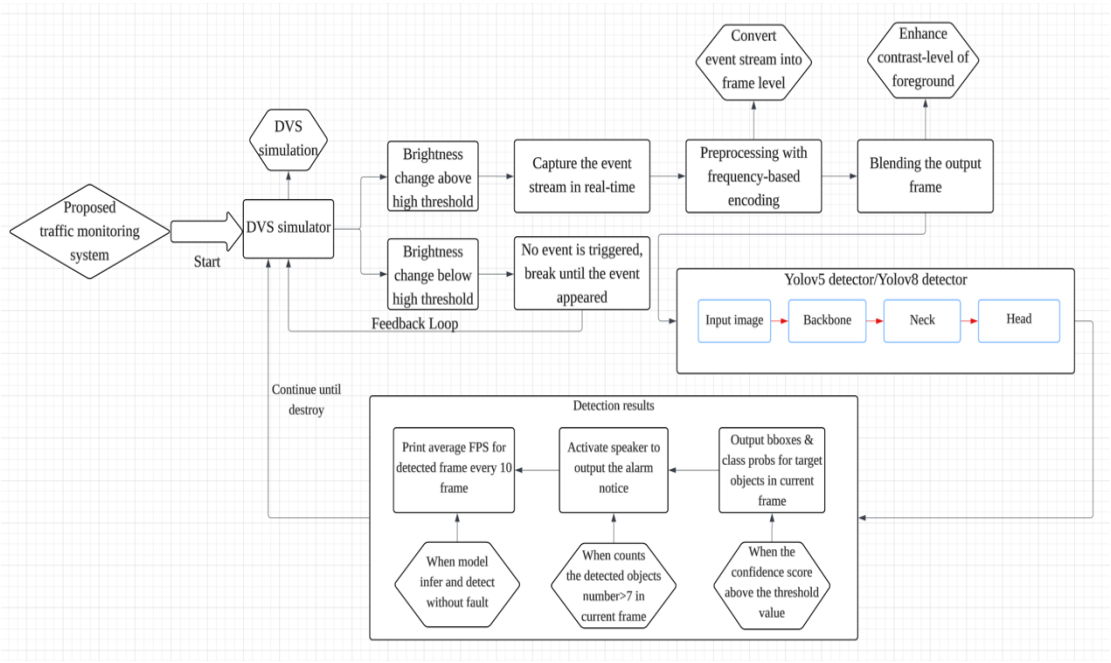


Figure 8: The draft of flow chart for the proposed system

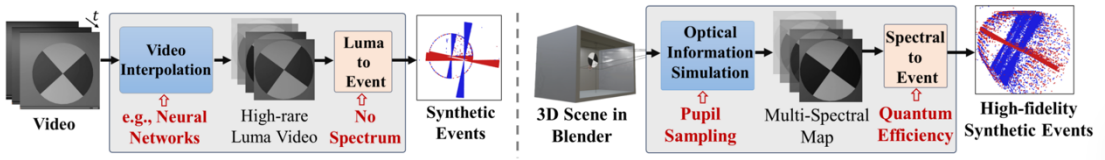
In Fig.8, the flow chart shows the operation logic theoretically. DVS is simulated by Python, there is a feedback loop to listen to the new event when the brightness changes between two adjacent events that are below the pre-set high threshold. Then, preprocessing the outputted event streams to meet the requirements as input to the YOLO detection model. Then the detection results will be output after inferring the YOLO model and having successfully done the detection work. The noticeable point is that the overall monitoring system keeps running until the designed program is suspended by the developer or destroyed by hardware.

Chapter 3 Methodology

Chapter 3 will present the algorithms and principles of the designed framework in more detail from a mathematical perspective. This chapter contributes to offering deeper insights into the implementation logic and the solutions to the issues mentioned in Chapter 2.

3.1 The method of DVS simulation

Event-based simulators are widely used to provide synthetic data at the software level. Software-based DVS simulators can be separated into two main categories, which are full-function DVS sensor simulation and video-to-event conversion (v2e).



(a) Existing video to events simulators (b) Physical-based event camera simulator

Figure 9: Two categories of event camera simulators [21]

As shown in Fig.9(b), the first one is to emulate the full analogue front-end of a real DVS sensor by modelling each hardware stage, including photodiode response, current-to-voltage conversion, amplification, comparator thresholding with hysteresis, and precise timestamp interpolation within a synthetic 3D scene. Under this method, the well-known simulators are DAVIS and ESIM. The former one is to generate event streams, intensity frames, and depth maps with high temporal precision through time interpolation in a synthetic scene, while the latter one is to extend DAVIS by offering an open-source platform for modelling camera motion in true 3D environments, producing events and comprehensive ground truth data [25]. Even though the advantage of high physical fidelity is to make the simulation results closer to the real DVS, the complexity and cost of implementation will increase significantly. What is more, the operation speed of the proposed system will be affected by lowering the FPS. Based on the considerations of higher speed and lower cost, another method called V2E is preferred to deploy. Fig.9(a) indicates that the V2E takes three-channel RGB videos directly as inputs that ignoring the process of modelling the real-world complete camera system. For the real-time object capturing, the input RGB frames can be converted into the grayscale form by only extracting the luminance channel of the up-sampled sequence [26]. The step of video interpolation is optional to effectively increase the FPS of the input frames. Based on the theoretical principles of v2e, this paper proposed a set of steps to obtain the DVS output in Fig.10.

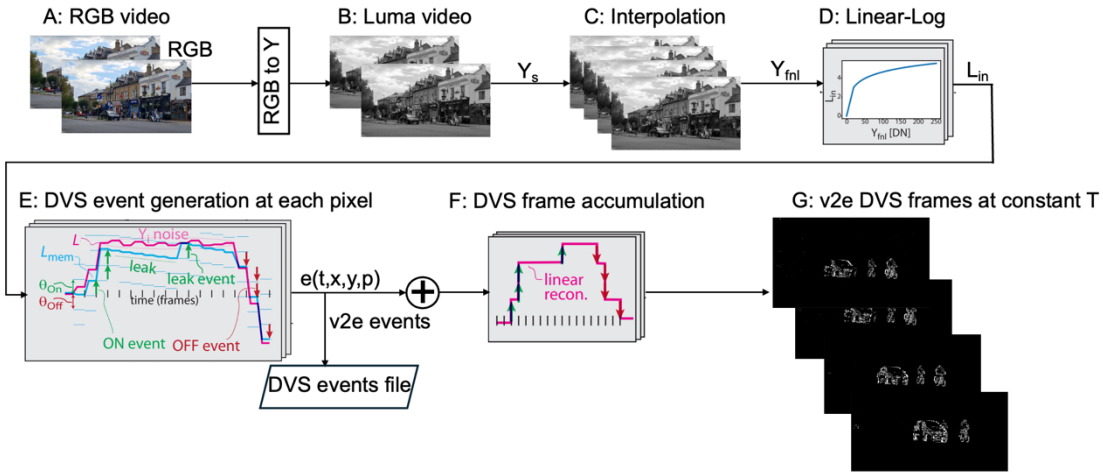


Figure 10: The proposed design of DVS simulation

The source input is the RGB video with T seconds in step A. The original RGB frames are converted into N luma grayscale frames to generate Y_s in step B, whose logic is expressed by the mathematical form in Equation 1. Each converted grey frame has a timestamp with $0 < t_i < t_j < t_N < T$. Then, the Y_s resolutions are increased to improve the FPS temporarily beyond the limitations of the source in step C.

$$Y_s = \{Y_s^{(i)}\}_{i=1}^{N} \quad (1)$$

$$U_{fnl} = \frac{1}{f_s \times C} \quad (2)$$

As shown in Equation 2, the upsampling ratio U_{fnl} is calculated with the parameters of source FPS and the maximum timestamp difference. U_{fnl} can be adjusted to upsample by controlling the ΔT_{max} to the minimum, it can be kept to 1 if the source FPS is large enough. The new frame rate after the step of interpolation can be obtained by Equation 3. The overall upsampled frames Y_{fnl} for all frames N from the luma video are measured by Equation 4.

$$f_{fnl} = f_s \times U_{fnl} \quad (3)$$

$$Y_{fnl} = \{Y_{fnl}^{(j)}\}_{j=1}^{N \times U_{fnl}} \quad (4)$$

For the next step of D in Fig.10, the transformation process of linear intensity to logarithmic intensity is implemented to mimic the DVS's logarithmic photoreceptor response. Because the intensity in conventional digital videos is treated linearly, while the DVS prefers to compare the intensity difference in logarithmic space to reduce the quantization noise [26]. The mathematical expression of converting linear to log of intensity is shown in Equation 5.

$$Lin(x, y) = Log_{10}\left(1 + Y_{fnl}(x, y)\right) = L_{mem} \quad (5)$$

In step E, two thresholds are defined with the absolute positive and negative values. As the difference between log intensity and luma intensity falls into any of the thresholds, then the event is triggered. The event can be output when the value of (L_{mem}-L) is greater than 0 to drop into the high threshold for triggering on event. The random noise of background cannot be avoided since the slight motion of DVS sensor is mimicked. Then the triggered event outputs each detected event e (t,x,y,p). As a result, the event streams are accumulated and output at a constant duration.

Algorithm 1: DVS Logical algorithm in Python

RGB Video loading:	ret, frame0 = cap.read()
Get RGB video source FPS:	f_s = cap.get(cv2.CAP_PROP_FPS)

Upsampling ratio by interpolation: U = max(1, int(np.ceil(1.0 / (f_s * C))))
Luma Y: L = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
Transfomation of linear-logarithmic: L_{mem}= np.log1p(L + 1)
The log intensity difference: delta = (L_{mem} – L)
ON events triggered coordinates: ys, xs = np.where(delta > +dvs_threshold)
buffer_events += list(zip(xs, ys, [1]*len(xs)))
OFF events triggered coordinates: ys, xs = np.where(delta < -dvs_threshold)
buffer_events += list(zip(xs, ys, [-1]*len(xs)))

Algorithm 1 shows the draft of logical implementation of the proposed DVS simulation in Python based on Fig.10, not the final version of the implementation of the DVS simulator work.

3.2 The methods of event-to-frame encoding

To address the issue of the conversion from event to frame, the pixels of continuous event streams can be encoded to a sequence of frames by three common encoding methods that are based on frequency, surface of active events (SAE), and LIF (leaky integrate-and-fire), respectively [27]. All of them do not affect the real-time detection speed a lot, even though the conversion process seems an extra heavy load task for the overall system.

3.2.1 Event-stream encoding based on frequency

Frequency-based encoding counts the number of events that occur at each pixel over a custom-defined fixed time interval, mapping the event frequency to an image intensity value. A higher frequency implies that the object is moving persistently within the sampling window to be converted with a higher intensity value. The setting of a shorter accumulation window can obtain finer temporal detail with noisier frames, while the longer one produces smoother images with more motion blur [19]. Therefore, there is a trade-off in setting the time intervals. The working principle is shown in Equation 6, which is a sigmoid-based normalization function. Applying this method can also increase the contrast level difference of the edges between the static objects and the moving objects. The comparison results of the difference between different duration of accumulation time will be displayed in Chapter 4.

$$\sigma(x) = 255 \times \left(\frac{1}{1 + e^{-\frac{x}{2}}}\right) \quad (6)$$

Where:

X is the number of events (ON + OFF) detected at a specific pixel during a

fixed time window,

σ(x) is the normalized pixel intensity output ranging between 0 and 255.

3.2.2 Event-stream encoding based on surface of active events

As mentioned before, the events e(t,x,y,p) are generated when the on threshold is achieved. The events change pixel with time when the motion state is changed. The second encoding method SAE records the timestamps of the most recent events per pixel, mapping the events by the axis of time on the 2D image [18]. As shown in Equation 8, the normalization of pixel intensity is defined by the difference between the latest timestamp of each event and the initial time. For more recent events, the normalized intensity of brightness will brighten to 1, decaying with time to form a long tail. Equations 7 and 9 show how it turns timestamps per pixel into a time surface image S(x,y;t) by using a decay function Δt at time t. As a result, SAE is a dense, fixed-size image that encodes exactly how recently each pixel changed, making it directly compatible with convolutional neural networks.

$$SAE: (x, y) \rightarrow t \quad (7)$$

$$SAE: (x, y) = 255 \times \left(\frac{t_p - t_0}{T}\right) \quad (8)$$

$$SAE: S(x, y; t) = \exp\left(-\left(\frac{\Delta t}{\tau}\right)\right) \quad (9)$$

Where:

τ is a time constant that controls how quickly old events fade

The small value of τ indicates that the fast decay of S from 1 to near 0, the older event vanishes almost instantly. It may fail to capture the slower or subtle motions. In contrast, the large value of τ shows that S remains high for a long period of time after an event. Even though more kinds of events can be captured, the motion blur may occur when quick motion happens. Therefore, the value of τ needs to be fine-tuned in the implementation stage.

3.2.3 Event-stream encoding based on leaky integrate-and-fire

In leaky integrate-and-fire (LIF) event-stream encoding, each pixel of the simulated sensor is modelled as a spiking neuron that integrates incoming brightness-change events over time while simultaneously leaking its internal state back toward a resting potential when no motion occurs [21]. As shown in Fig.11 [27], every time a new ON or OFF event arrives at pixel (x,y), the corresponding membrane potential is incremented by an input weight. It decays exponentially with the time constant τ when no input is coming.

When Vx,y(t) crosses a predefined firing threshold, the pixel emits an output spike and its potential is reset to a lower value, optionally entering a brief refractory period. This mechanism produces an asynchronous, sparse stream of output events. Rapidly moving edges of objects generate sufficient input to drive the potential over threshold, while slowly varying or noisy fluctuations tend to leak away without spiking.

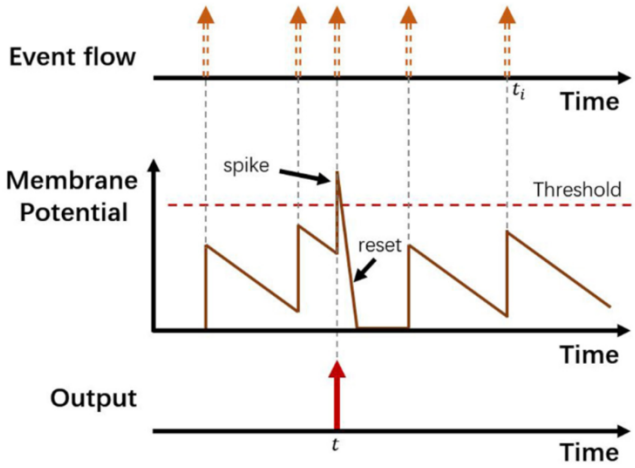


Figure 11: The working mechanism of LIF [22]

3.3 The method of improving the contrast of foreground

To integrate both appearance and motion cues into a single input for our YOLO-based detector, an alpha-blending fusion strategy is employed that combines the DVS-frequency frame and the color-coded event polarity map into a standard three-channel BGR image. Concretely, the per-pixel frequency-based encoded image is computed by counting ON/OFF events over a sliding window and normalizing to [0–255]. An RGB polarity mask is proposed to be generated by defining green pixels for ON-events and red for OFF-events of the same dimensions. Using OpenCV's cv2.addWeighted(src1, α, src2, 1–α, 0) function with weights α = 0.7 for the frequency base and 0.3 for the polarity overlay, then a fused frame can be produced that retains the structural and textural details of the scene while superimposing high-contrast motion edges. This early fusion approach offers several advantages for the performance of designed system. First, by keeping the output as a conventional BGR image, no modifications to the YOLOv5 and YOLOv8 architectures are needed. The network ingests the fused frame just as it would a regular camera image. Second, the choice of α balances the trade-off between static context and dynamic saliency. A higher weight on the frequency map preserves background features essential for accurate classification, while the motion overlay attracts the detector's attention to rapidly moving or low-contrast objects [28]. Third, the YOLO backbone's convolutional filters can learn to jointly exploit texture, shape, and motion signals in a

single forward pass since the fusion is performed at the pixel level prior to any feature extraction, reducing inference latency compared to multi-stage fusion schemes. In implementation, the blending step is immediately after event accumulation and frequency-map generation inside the real-time processing loop. Each blended frame is then fed directly to the TFLite interpreter for bounding-box regression and class probability estimation. Empirically, this fusion pipeline enhances recall on fast-moving targets and in challenging lighting, while maintaining precision on static objects, demonstrating that simple linear blending of DVS-derived motion information with appearance data can substantially improve detection robustness without adding computational or architectural complexity [29].

3.4 The method of dataset collection with the custom annotation

With the development of the DVS-related object detection applications in recent years, there are some existing real-world labelled open DVS datasets on GitHub for free use and download, such as EventVOT and eTraM [15]. EventVOT dataset captures the high-resolution data of vehicles and pedestrians on the street across various motion speeds and lighting conditions, while eTram dataset also captures more than 10 hours of data of eight traffic participants. The key point is that both collect data with the Prophesee EVK4 HD camera, and experts have annotated the data. The synthetic DVS datasets are also developed with software simulators like SEVD, Event-KITTI and ESfP-Synthetic [17]. The characteristics of lower operation cost and time consumption make the synthetic dataset have more classes of traffic participants compared with the real DVS camera. However, even though the kinds of datasets can provide enough annotated DVS event-based data for training with YOLO detector, the complementary work of DVS datasets is still required to diversify the DVS datasets. In this paper, the high-resolution RGB videos of cars, pedestrians, motor cyclists and cyclists will be collected and freely downloaded from Google's Pexels official website.

The process of data annotation is important to convert raw data into valuable resources for artificial neurons to learn in the algorithms of deep learning and machine learning. The quality of annotation can directly affect the performance of the classification and object detection tasks. In Table 1, the characteristics of mainstream manual annotation tools to support image labelling are presented clearly [30]. There are some reasons to evidence that the LabelImg labelling tool is the best choice compared with VGG Image Annotator (VIA) and Brat Rapid Annotation Tool (BRAT). Firstly, LabelImg outputs the YOLO compatible XML format directly without an excessive conversion step to output the bounding boxes positions and class labels based on a light-weight resource,

which conforms to this paper's original intention of low cost and the training requirements of YOLO. Secondly, it is simple to use with a fast speed to define classes and bounding boxes, which reduces the complexity of the data annotation process. Even though VIA can draw bounding boxes with multiple shapes to flexibly mark the contour more precisely, it is not specialized as LabelImg to export XML format data and support various shortcuts and controlling functions to optimize the user's experience. The BRAT is not focused on video or image annotations, it prefers to be used for natural language processing to annotate text contents efficiently.

Table 1: The characteristics of Mainstream manual annotation tools

Annotation tool	Key features	Applications	Pros / Cons
LabelImg	Bounding boxes, XML/TEXT format output	Fast, light-weight annotations. Mainly for annotating bbox	Easy to use, cross-platform compatibility/ Limited functions
VIA	Multiple shapes of bbox	Mark the contour of objects	Flexible / no XML format export
BRAT	NLP, custom attributes	Annotate text rather than image	High flexibility, less efficient

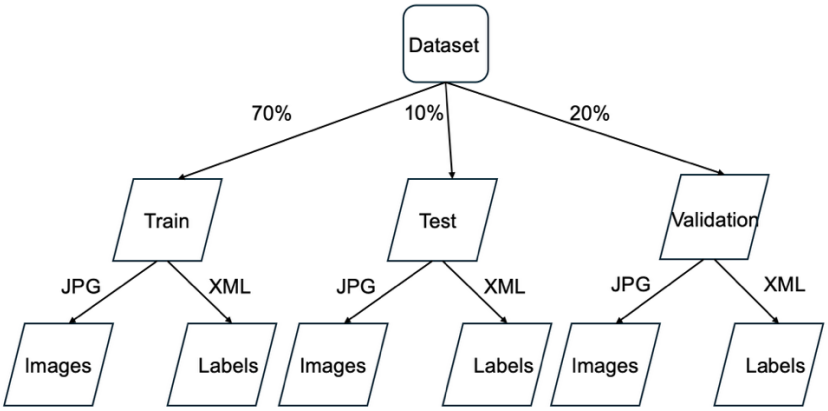


Figure 12: The structure and configuration of the custom dataset

As shown in Fig.12, the dataset is composed of three parts with a weight ratio of 7:2:1, including train set (70%), validation set (20%) and testing set (10%). Under each of the subsets, the JPG images and XML labels are arranged separately. The aim is to organize the collected dataset to prepare for training without misunderstanding by machines.

Algorithm 2: Activating LabelImg in Python on Wins Anaconda

```
# 1. Install LabelImg (and its Qt5/LXML dependencies)
python3 -m pip install --upgrade pip
python3 -m pip install pyqt5 lxml
```

```
python3 -m pip install labelImg
# 2. Launch the LabelImg GUI
python3 -m labelImg
```

Algorithm 2 presents the Python code to install the LabelImg annotation tool with its required dependencies to activate it. LabelImg is built on the Qt framework for its graphical user interface, and PyQt5 provides the Python bindings for Qt 5. Installing pyqt5 supports all the widgets, windows, dialogues and event loop running on LabelImg's GUI without mistakes. Without PyQt5, the application cannot render its interface or respond to mouse or keyboard events.

3.5 The CNN-based YOLOv5 and YOLOv8 model

The detection models in this paper are determined to deploy YOLOv5 and YOLOv8, respectively. YOLOv5 was developed by the Ultralytics company based on the PyTorch framework in 2020 while YOLOv8 was explored successfully based on the YOLOv5 framework with several improvements to the architecture and the developer experience.

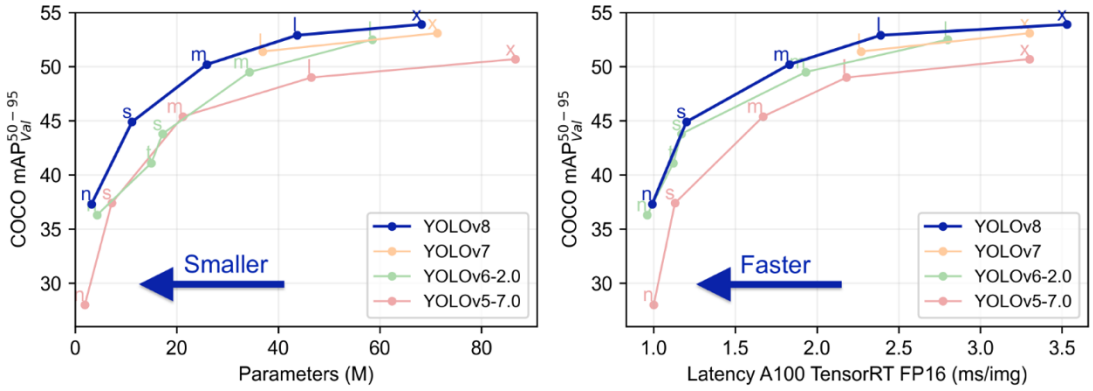


Figure 13: The theoretical performance of the YOLOv5 and YOLOv8

As shown in Fig.13, both of them have five models, including the versions of nano, small, medium, large and extra-large. Each version is built based on the different requirements of the system design. The detection results are obtained by testing the COCO128 dataset on an A100 GPU. There is a trend to improve the average precision by applying a larger model size. For the extra-large version, it is suitable for the design that requires satisfying the high standard of detection accuracy without the concerning for the deployment cost of hardware. The detection speed is highly dependent on the performance of the hardware and the size of the YOLO model. This paper's proposed real-time monitoring system requires low latency with high detection speed to operate the YOLO model while tolerating sacrificing a little accuracy. Therefore, the small and nano versions are preferred to deploy in the vehicle-mounted system. The collected datasets are compatible with both models, and the training processes are very similar. In

chapter 4, all model sizes are trained with the same parameters and compared their performance under the same conditions.

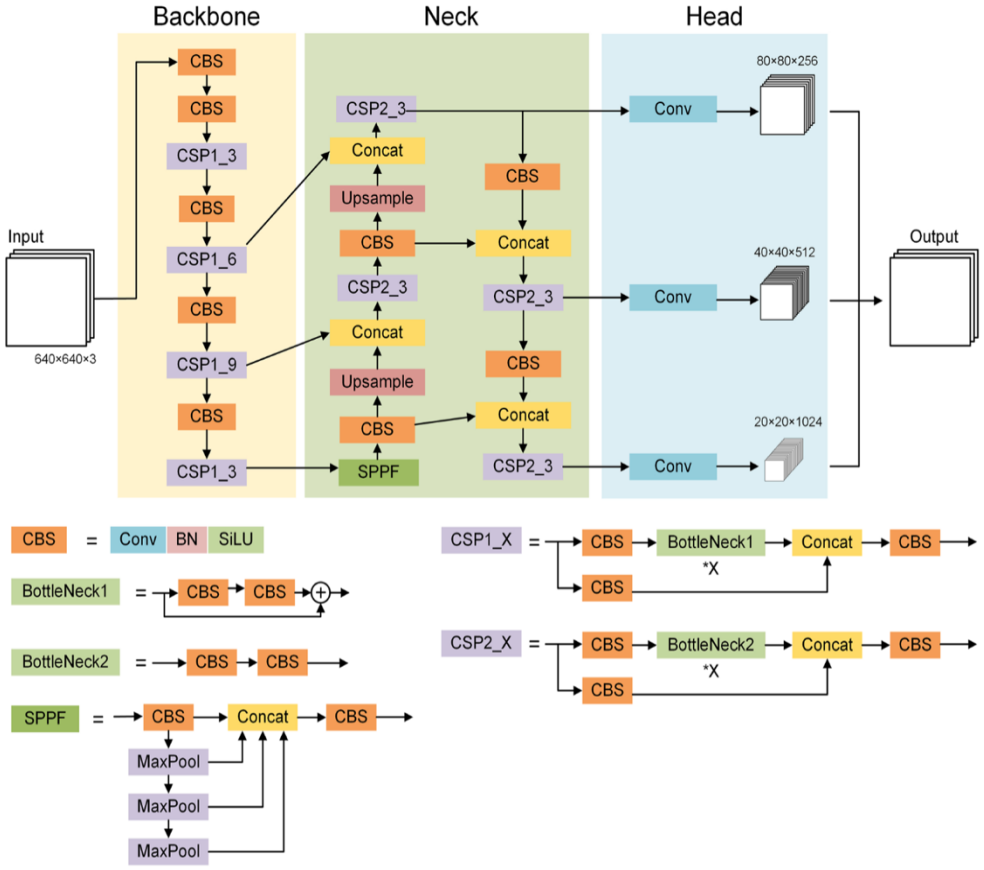


Figure 14: The structure of YOLOv5

The structure of YOLOv5 is demonstrated in Fig.14 [31], the framework consists of three parts, including backbone, neck and head. The Backbone of YOLOv5 is targeted for hierarchical feature extraction from the input image of 640 × 640 × 3. It begins with successive CBS blocks that are formed with Convolution, BatchNorm and SiLU activation, which downsample the spatial resolution and increase feature depth after extracting the feature by convolution layers. The interleaved modules in the backbone part are CSP1_X modules, which are the cross-stage partial connections with X Bottleneck1 blocks, splitting the feature map into two parts. It routes one part through a sequence of Bottleneck1 sub-blocks, and then concatenates it back with the shortcut path. This design reduces computational redundancy and gradient information loss while preserving a rich diversity of learned features. Specifically, the network employs a CSP1_3 block at the first scale, followed by alternating CBS and CSP1_6 modules to deepen the representation, and combined with a final CSP1_3 to produce three multi-resolution feature maps. After the Backbone, the Spatial Pyramid Pooling Fast (SPPF) layer further enriches the most abstract feature map by aggregating context across

multiple receptive fields. SPPF applies three successive max-pooling operations with increasing window sizes in parallel, concatenates their outputs, and processes the result through another CBS block. This module captures fine-grained and global context, making the detector more robust to scale variations and enabling it to recognize objects of widely varying sizes.

The Neck section implements a top-down and bottom-up information fusion strategy inspired by the Path Aggregation Network (PANet). It begins by applying CBS to the deepest Backbone feature map, then upsamples it by a factor of two and concatenates it with the intermediate Backbone output by a Concat operation. The combined feature is processed through a CSP2_3 module that has two parallel CBS streams feeding into three Bottleneck2 blocks before concatenation, yielding a refined high-resolution feature map [32]. This map is again upsampled and fused with the shallowest Backbone feature in an identical manner from CBS to Upsample to Concat to CSP2_3, therefore, propagating semantic information to earlier layers and preserving localization cues. What is more, the Neck employs a symmetric bottom-up path that is parallel to these top-down branches, including the step of downsampling the fused high-resolution features via CBS with a stride of 2, concatenating with the intermediate feature maps, and passing through another CSP2_3 block, respectively. This multi-scale bidirectional fusion enhances the detector's sensitivity to both small and large objects, effectively merging low-level spatial details with high-level semantic abstractions.

Finally, the Head of YOLOv5 takes the three multi-scale feature maps output by the Neck at resolutions 80 × 80, 40 × 40, and 20 × 20 with channel depths 256, 512, and 1024, respectively. And it applies a simple CBS-Conv sequence to each before producing parallel bounding-box and class-probability predictions. Each head uses a 1 × 1 convolution to reduce dimensionality and then a 3 × 3 convolution to aggregate local context, culminating in a final 1 × 1 convolution whose output tensor has the shape [grid_h, grid_w, anchors × (5 + num_classes)]. Here, the number '5' corresponds to the four bounding-box offsets and one detection score, while 'num_classes' refers to the car, rider and pedestrian in our application. During inference, these raw predictions are decoded, and non-maximum suppression is applied to yield the final set of detections.

Across the entire framework, YOLOv5 employs SiLU for smoother gradient flow and faster convergence. The mathematical expression for the activation function of SiLU is shown in Equation 10.

$$SiLU(x) = \left(\frac{x}{1 + e^{-x}}\right) \quad (10)$$

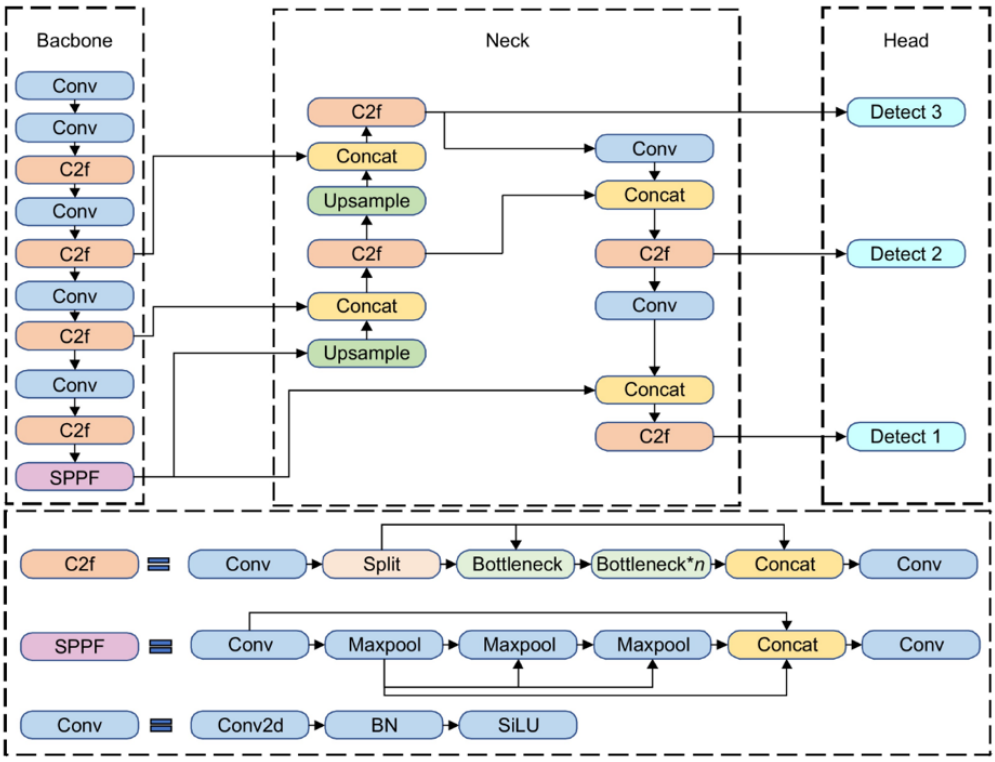


Figure 15: The structure of YOLOv8

YOLOv8 is a more advanced version based on the YOLOv5 framework. It applies a more unified, building block called C2f alongside an anchor-free, decoupled detection head. YOLOv8 replaces CSP1_X and CSP2_X in YOLOv5 entirely with C2f, which is a single module that begins with a 1×1 CBS entry convolution, splitting the resulting feature map into two paths [33]. Then passing through one of two paths through a chain of lightweight Bottleneck blocks. After that, the bottleneck output is concatenated back with the untouched split. A final 1×1 CBS exit convolution fuses these channels, supporting the same capacity as CSP with much simpler control flow and fewer branches. This change appears both in the Backbone where YOLOv8 stacks three C2f modules at progressively deeper scales, and in the Neck where it implements top-down and bottom-up PANet-style feature fusion. YOLOv8 also retains the SPPF layer from YOLOv5 at the end of its Backbone, ensuring that receptive fields of varying sizes are aggregated before feature fusion.

In the Head part of YOLOv8, each scale's feature map first passes through a small CBS block, then divides into two parallel branches to predict box sizes and class probabilities. Finally, a unified Detect module is applied to dynamically match ground-truth boxes to feature-map points at training time, dispensing with the need for preset anchor boxes altogether. This design not only reduces hyperparameter tuning but also improves the recall rate on objects of unusual scales. In contrast, YOLOv5 employs an

anchor-based head that predicts bounding-box offsets and class confidences in a single fused tensor per scale, relying on manually configured anchor priors and requiring complex decoding during inference. However, non-maximum suppression is required to be performed on these raw predictions to optimize the bounding boxes of detection results for both YOLOv5 and YOLOv8 to suppress the low-confidence boxes.

During the training process, there are some important hyperparameters that need to be adjusted to optimize the model training results, such as batch size, epoch and learning rate. An epoch represents one complete pass of the entire training dataset through the network during the training process. Essentially, it's how many times the algorithm sees the entire training data. Each epoch involves forward and backward passes for each batch of data.

In convolution neural networks, the training data is divided into smaller batches. Then each batch is processed in a forward pass to make predictions and then adjusts its internal parameters of weights through backpropagation in a backwards pass [34]. The batch size is defined as the number of samples used in each of these batches during the training process. There are three types of results of gradient descent trends for cost against the number of iterations that are highly dependent on the setting of batch size, which are shown in Fig.16. The batch gradient descent (BGD) is defined the batch size to the entire numbers of the collected training set, which generates a smooth non-linear descending line. Applying BGD can update the weights stably with less noise since the training loss is averaged on the whole dataset. However, the large dataset leads a heavy burden on the memory. And the training results can be poor due to the infrequent update of new weights. The stochastic gradient descent (SGD) processes only one sample at a time for each update, making the batch size equal to 1. This method provides the characteristic of fast convergence and continuous feedback for the training progress. But the problems of longer time for training and frequent noisy updates are also need to be considered. In contrast, the mini batch gradient descent (MBGD) produces a smoother cost function compared to SDG. It processes a pre-defined a smaller number of samples at a time compared with BGD so that MBGD is more scalable and more frequently updating than BGD. It also occupies less memory load than SGD. However, selecting the optimal batch size requires to evidence by experimentation. The method is that to try different batch sizes and monitor the model’s performance in terms of training speed, convergence, and generalization ability.

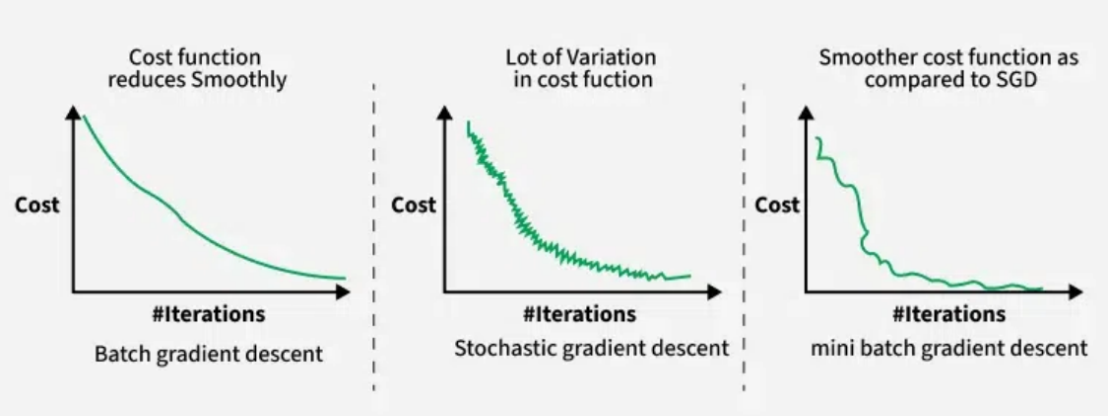


Figure 16: Three types of gradient descent for different batch sizes selection [34]

The learning rate is a hyperparameter that determines the step size at which the model adjusts its weights during training. It controls how quickly the model learns from the data. The learning rate is a small positive value that typically between 0 to 1. It is multiplied by the gradient of the loss function during each training iteration to determine how much the model's weights are adjusted based on the error evaluated from the dataset. Fig.17 demonstrated that a high learning rate can lead to faster initial progress but also causes the model to overshoot the optimal solution and oscillate, while a low learning rate can result in slow convergence and potentially get stuck in a local minimum value. A near-zero learning rate cannot be able to learn any features in networks. Since the learning rate exceeds a threshold value, it starts to learn with a small loss rate.

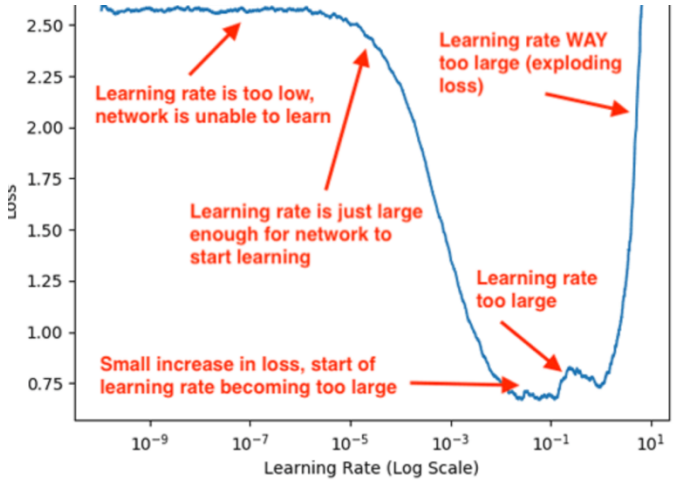


Figure 17: The effect of learning rate on loss function [35]

As shown in Figure 18, the performance of training can be evaluated by analyzing the graphs of validation accuracy and training loss [35]. As the applied number of epochs increases, the over-fitting phenomenon appears when the validation accuracy drops from the highest point or when the training loss increases from the bottom. The final results

miss the best updated weights due to training with more epochs than just-fitting. Oppositely, as the number of epochs are not sufficiently deployed to train the model, the under-fitting phenomenon can be caused to output a poor validation results. The target objects in validation set cannot be recognized correctly and distinguished from the background as the trained model learning few features during the training process. The under-fitting model is exported when no major features are learnt successfully.

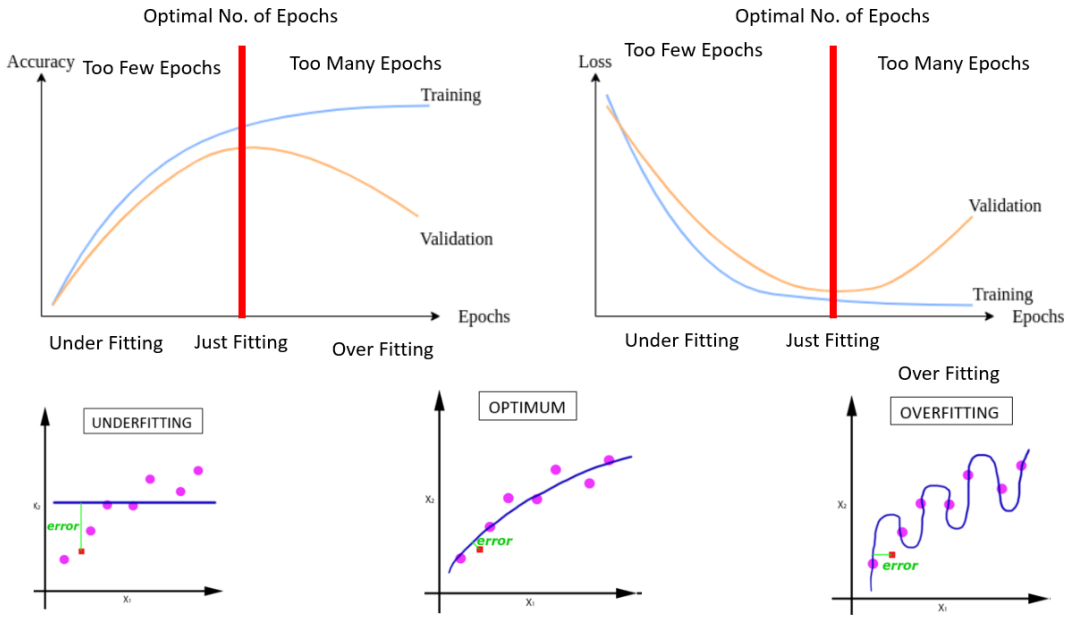


Figure 18: The theoretical evaluation of training results

In conclusion, the hyperparameters are required to adjust based on every time's results of training and validation. No standard answer can be obtained from the first-time training. The adjusting of hyperparameters is dependent on the collected datasets without a uniform value to improve the model performance. The process of collecting and annotating dataset's quality and quantity can also affect the validation results even though professionally adjust the training process without the problem of under-fitting and over-fitting.

3.6 Post-processing with non-maximum suppression

Non-Maximum Suppression (NMS) is used to eliminate redundant bounding boxes around the same object. During inference, a trained YOLO detector generates multiple overlapping boxes with various confidence scores for a single target. NMS removes all overlapped boxes that occupy the low-level confidence score to only remain the highest-confidence box at each location. First, it sorts all candidate boxes by descending confidence score. Then, it retains the top-scoring box and suppresses any other boxes whose Intersection-over-Union (IoU) with that box exceeds a chosen threshold [29]. For

example, a IoU threshold of 0.5 means the geometric area of two boxes overlapped is 0.5. The logical expression of IoU is expressed in Equation 11. Next, it moves to the highest-scoring box among them and repeats the suppression until no candidates remain. When applied per class, it yields a final set of non-overlapping detections that balances precision and recall; the filtered boxes with their retained confidence scores and class labels form the basis for the reported metrics.

$$\left(\frac{area(B1 \cap B2)}{area(B1 \cup B2)}\right) \geq threshold \quad (11)$$

Chapter 4 Experiments

In Chapter 4, the experimental implementations of the design will be conducted based on the proposed theoretical method that was mentioned in Chapter 3.

4.1 DVS simulation

The sub-section of 4.1 provides a clear description of the implementations of converting the original RGB video into the event stream with a visual view, three methods of event stream encoding based on frequency, SAE and LIF, and blending rendering.

4.1.1 v2e conversion

An original RGB video is collected to recognized as the test video from Google iStock free sources in the design process, which contains the target objects of cars, pedestrians and riders. Then the proposed v2e conversion process is applied to simulate DVS with the step of luma video to grey scale, interpolation of grayscale video, linear to logarithmic conversion, event capturing with the pre-defined high threshold, generating DVS frame accumulation and outputting the event stream with a CSV file.

1. 10th frame

2. 20th frame

3. 30th frame

4. 40th frame

5. 50th frame

6. 60th frame

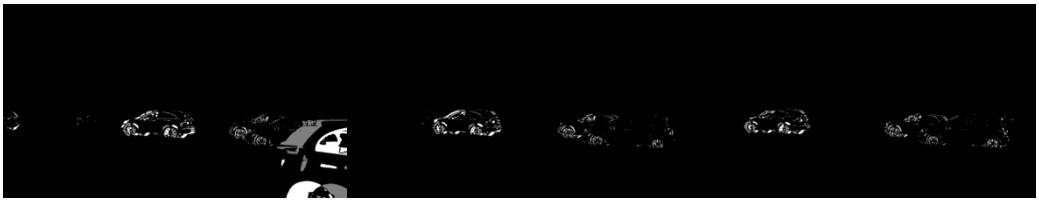
7. 70th frame 8. 80th frame 9. 90th frame

Figure 19: 9 frames of original RGB test video

In order to observe the changes among captured frames more clearly, the frames of 10th, 20th, 30th, 40th, 50th, 60th, 70th, 80th and 90th are sampled to present in Fig.19, which demonstrates the captured 9 frames of the collected RGB test video. Each frame is filled with a lot of unnecessary background information, such as buildings, trees and traffic lights. Without converting the RGB image into event data, the performance of target detection would be greatly affected by unless information. Based on the proposed method of v2e, DVS is simulated successfully with the steps of scaling the RGB video to grayscale, interpolation of grayscale video, the intensity from linear to log conversion, event generation and event accumulation. The implementation code will be shown in the Appendix. As mentioned before, there are two pre-defined thresholds during the event generation step to make an adjudgment of ON and OFF events, which are 0.8 and -0.8, respectively. When the change in log intensity for each pixel between the current frame and the previous frame is above 0.8 or lower than -0.8, the event is triggered and recorded.

1. Events in 10th frame 2. Events in 20th frame 3. Events in 30th frame

4. Events in 40th frame 5. Events in 50th frame 6. Events in 60th frame



7. Events in 70th frame 8. Events in 80th frame 9. Events in 90th frame

Figure 20: 9 frames of event data

As shown in Fig.20, the simulated neuromorphic sensor only captures the moving objects like cars when the lighting intensity exceeds the high threshold. In event-based vision, ON and OFF events represent changes in pixel brightness. During the implementation, ON events are assigned a pixel value of 255, which corresponds to the representation of white color in an 8-bit grayscale event stream, making them highly visible for moving objects that enter a new pixel. OFF events are assigned a mid-grey with the value of 127 to indicate the state of moving objects leaving the last pixel, which provides a clear visual contrast against both ON events and background pixels. The choice of 255 and 127 for ON and OFF events is a deliberate and practical design to enhance the readability of event frames without altering the underlying data. The most important thing to be mentioned is that the visualized figures shown in Fig.20 contain thousands of events, rather than the description of each image presents one event. Each image is the result of the accumulation of lots of events to form the visualized view. For example, the first 10 generated event streams e(x,y,t,p) in the 10^{th} frame are shown in Table 2. The event is generated only when changes of intensity exceed the high threshold compared with the 9^{th} frame.

Table 2: The partial simulated DVS Event Stream in the 10^{th} frame

x	y	timestamp	polarity
120	536	39.139	-1
121	536	39.139	1
122	536	39.139	1
234	541	39.139	1
235	541	39.139	1
236	541	39.139	1
237	541	39.139	1
238	541	39.139	1
240	541	39.139	1
241	541	39.139	1

Based on the generated event stream in the 10^{th} frame, two three-dimensional plots are produced based on all event streams of the 10^{th} frame of the test video and all generated event streams in the test video, respectively. As shown in Fig.22(a), the timestamp is unified with 39.139ns at the 10^{th} frame; there are thousands of events presented by the scatter points in the coordinate system in plane. However, when all frames' events are considered in the 90s test video, huge numbers of events are produced. However, not all recorded events are useful in this proposed design of traffic monitoring

for only cars, pedestrians, motorcyclists and cyclists. In the final integration part, the DVS only outputs the target event based on the target objects when a detection confidence score around them is higher than a threshold that is determined based on the F1 curve.

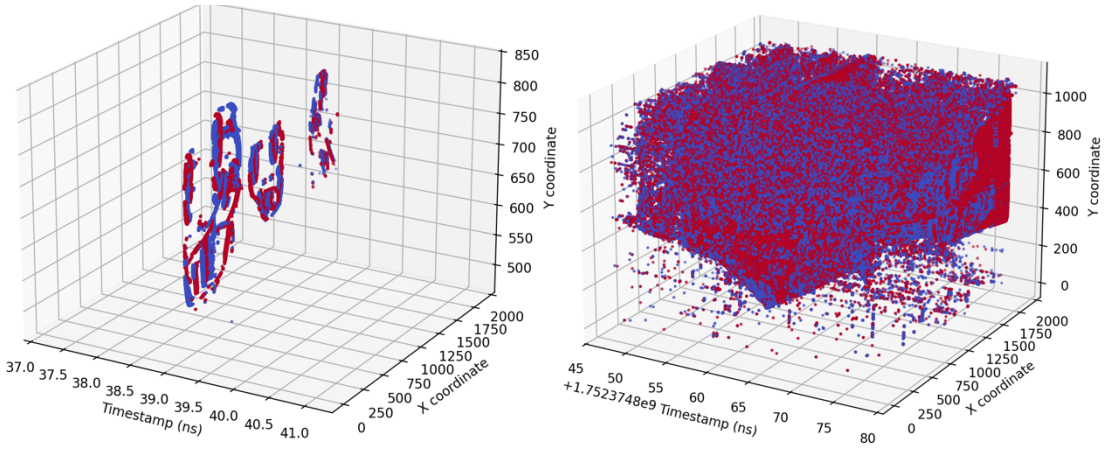


Figure 22(a): The 3D plot of events for 10th frame of test video

Figure 22(b): The 3D plot of events for all frames of test video

4.1.2 Preprocessing with encoding event-streams based on frequency, SAE and LIF

After generating the event-based data, the next step is to convert the event streams to a frame-based level with the encoding method to obey the requirements of YOLO detection model training. Fig.23 shows the results of encoding event stream based on frequency; there is no comparison of logarithmic intensity difference, as in the same way as the original DVS frame. All motions are recognized as linear moving states, meaning that it is easy to see how frequently a motion persists to move rather than understanding which motion is happening more recently, like the method of SAE. However, since the frequency-based method accumulates events uniformly within the defined sampling window, it does not differentiate between newer and older events within that period. This can result in slight motion blur, especially when objects move quickly or when the accumulation window is longer, as past events linger with equal weight. Despite this drawback, the frequency-based encoding method remains intuitive and computationally simple, making it a practical option for detecting motion regions and extracting temporal features when precise timestamp information is less critical.

1. Encoded frame-level events in frames of 10^{th}, 20^{th} and 30^{th} based on frequency

2. Encoded frame-level events in frames of 40^{th}, 50^{th} and 60^{th} based on frequency

Figure 23: Event-streams encoding based on frequency

The accumulation time is also a factor to affects the process of capturing target features, as shown in Fig.24. Smoother features can be caught at a longer accumulation time with 0.5ns, and the edge of the moving object can be drawn more obviously. However, the motion blur cannot be avoided, which can also increase the difficulty of classification. The finer detail can be obtained when accumulated at a shorter time with 0.01ns, and the phenomenon of motion blur and noise is also minimized. Therefore, the accumulation time with 0.01ns is a better choice to capture the targets.

Figure 24: Frequency-based encoding under accumulation time of 0.01ns (Left) & 0.5ns (Right)

Another encoding event-stream method is based on Surface of Active Contents (SAE), which is more complex to be able to obtain more information than the former one. The implementation results of SAE are demonstrated in Fig.25 on the simulated DVS, as shown clearly, the moving cars in this test video demonstrate different brightness due to the newer events leading the older events to decay with time. The reason is that the newer events are highlighted with a brighter intensity since their timestamps are closer to now, while the older events are faded exponentially with a darker intensity. Applying SAE can track fast changes, get the information of the moving direction based on the change of intensity and distinguish the timestamps of events effectively. In addition, this method also increases the contrast level of the foreground against the black background. The

output demonstrates how the SAE efficiently highlights the freshest contours of moving objects, enabling a clear distinction between active motion and static regions. This results in sharper edges and trails that accurately represent the direction and speed of motion over short time intervals. Unlike the frequency-based method, which treats all accumulated events equally within the window, the SAE naturally prioritizes the temporal recency of each event, suppressing background noise and irrelevant static details. However, it can still retain slight motion blur if the event density is high and the decay time constant is large.

1. Encoded frame-level events in frames of 10^{th}, 20^{th} and 30^{th} based on SAE

2. Encoded frame-level events in frames of 40^{th}, 50^{th} and 60^{th} based on SAE

Figure 25: Event-streams encoding based on SAE

By applying the method of SAE, the parameter of the decay time constant for the older event is required to be considered. There is a trade-off to adjust the accumulation time. The implementation of SAE with the accumulation time of 0.01ns and 0.5ns in Fig.26 indicates that the moving state and moving direction can be maintained clearly when adjusting the decay time to 0.5ns, but the motion blurs and the background noises are obvious. When adjusting the decay time to 0.01ns, the target object 'cyclist' is captured clearly without much loss. To sum up, the shorter accumulation time is more suitable for doing a real-time object detection task since the trained detection model can infer the detection results more robustly and more precisely.

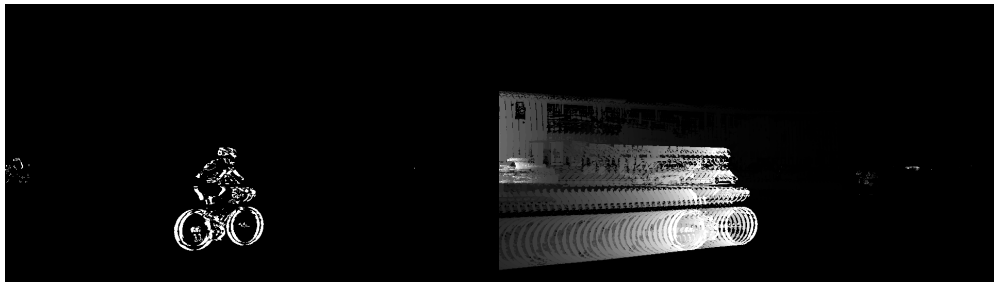


Figure 26: SAE-based encoding under accumulation time of 0.01ns (Left) & 0.5ns (Right)

The third method of event streams encoding is based on the leaky integrate-and-fire (LIF). In this method, each pixel functions as an artificial neuron that integrates incoming brightness changes over time and continuously leaks its membrane potential with each frame update. When the accumulated potential crosses the defined threshold of 0.8, it has the ability to avoid recognizing the noise or background as events. An event is triggered and the potential leaks again, allowing the simulation to capture not only the edges of moving objects but also their temporal continuity. The resulting output frames are illustrated below in Fig.27, demonstrating that the LIF mechanism effectively highlights sharp motion edges while suppressing background noise and stationary regions. Compared with the frequency-based encoding, the LIF-based frames preserve more recent and salient motion cues with significantly less motion blur because older events naturally decay through the leak process. The clearer object outlines can better reflect the sparse, asynchronous nature of real DVS output.

1. Encoded frame-level events in frames of 10^{th}, 20^{th} and 30^{th} based on LIF

2. Encoded frame-level events in frames of 40^{th}, 50^{th} and 60^{th} based on LIF

Figure 27: Event-streams encoding based on LIF

In the implementation of the LIF encoding, adjusting the decay factor directly influences the temporal persistence of the accumulated membrane potential at each pixel, which in turn shapes how motion and edges appear in the final DVS frame. Unlike the output from the former two methods, the output frame with a very low decay factor of 0.01 shows that the background is almost entirely black with very few visible structures, meaning that the potential leaks are too quickly for most transient events to reach the firing threshold of 0.8, leading to the loss of valuable motion detail and under-representation of target objects. The first output frame in Fig.28 with a decay factor of 0.2 achieves a better balance between temporal persistence and noise suppression, the moving objects are clearly outlined with less residual blur, and the background remains darker. The second output frame is generated with a relatively high decay factor of 0.5,

demonstrating more obvious motion trails and brighter accumulation because the pixel potentials decay slowly. Therefore, tuning the decay factor appropriately is crucial to highlight meaningful motion cues while minimizing ghosting artefacts. Overall, the results illustrate that a decay factor that is too high will retain redundant event history and blurred trails, while a too low decay factor will cause information loss. In conclusion, moderate decay values around 0.1 to 0.3 are effective in preserving crisp motion contours while maintaining the sparse, event-driven nature of the DVS representation.

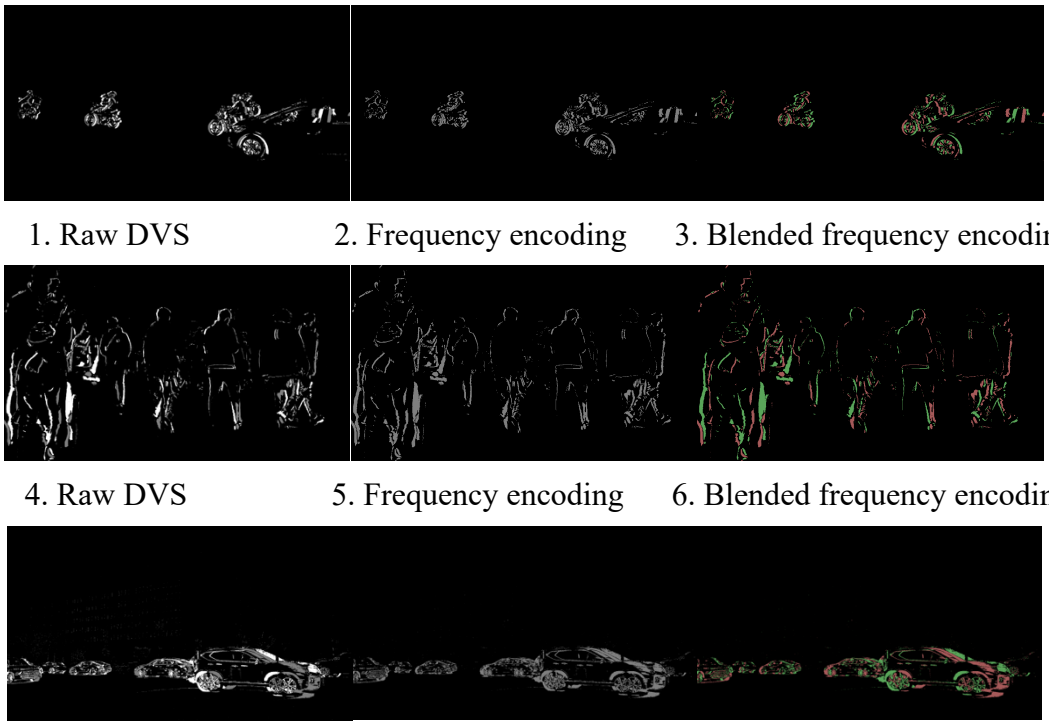
Figure 28: LIF-based encoding under accumulation time of 0.2ns (Left) & 0.5ns (Right)

In this paper, frequency-based event stream encoding was selected as the most suitable input representation for training the YOLO detection model. Compared to SAE and LIF, the frequency-based method offers a stable and dense frame format by accumulating event counts over a short, fixed temporal window. This approach provides a good balance between noise suppression and the way of consistent object highlighting, resulting in clear and continuous contours for moving objects. SAE can be too sparse and prone to information loss when motion is limited, the frequency encoding ensures that enough spatial features are present for the YOLO network to learn robust object boundaries. Moreover, frequency frames avoid the parameter sensitivity, which can suffer from under-integration or over-integration depending on the decay factor. By emphasizing the overall density and spatial coherence of motion events, the frequency-based method reduces the risk of missing partial object details and supports better generalization across varying object speeds. Therefore, frequency encoding is considered the optimal choice to deliver consistent, noise-resilient input frames that align well with the training requirements of the YOLO-based object detection model.

4.1.3 Blending encoded frames

By following the flow chart in Chapter 3, the next step is to blend the encoded event frames in order to increase the contrast level of the moving objects and map the DVS event frames with RGB color rendering. Analyzing the advantages of applying blending to the aspects of YOLO model training is valuable research, which is conducted by

comparing the output with raw DVS events and frequency-based encoding. The implementation results with the target objects are shown in Fig.29, including cars, pedestrians and cyclists. The raw DVS events are visualized at 1, 4 and 7, which lack of ability to continuously capture the moving objects' features, as the parts of the moving objects are recognized as background or noise when the brightness changes below the threshold of 0.8. Implementing the blending function on the frequency-based event streams encoding enhances the finer details to build sharper and clearer complete contours with the green and red rendering. Applying this method can be used to distinguish two adjacent or overlapping instances of moving objects directly. For training a YOLO-based detection model, this richer representation provides more discriminative features for the network's convolutional layers to learn, improving the model's ability to localize and classify moving objects under low-texture or complex motion conditions.



1. Raw DVS 2. Frequency encoding 3. Blended frequency encoding

4. Raw DVS 5. Frequency encoding 6. Blended frequency encoding

7. Raw DVS. 8. Frequency encoding 9. Blended frequency encoding

Figure 29: The visualize view of blending effect

4.2 Dataset construction

Even though there are many public DVS datasets issued and supported to use freely, the datasets purely for cars, pedestrians, motorcycles and bicycle cyclists are still scarce. In this paper, a custom high-quality DVS dataset is constructed based on the methodology in Chapter 3, following the order of collecting RGB video, v2e conversion,

Frequency-based encoding, and blending. The output frames that obey this order are necessary to act as the input for YOLO training in this designed system.

4.2.1 Dataset collection

In order to recover the real traffic scenes, 110 high-quality videos about 3 hours recording of the congested crossroads, bustling streets, pedestrian commercial streets, subway entrances and exits, and the narrow lanes of small towns were collected by downloading on the Google Pexels website (https://www.pexels.com/). Then the collected RGB videos are processed by the proposed method to convert into the event streams with the format of frame-level plus blending effect. As is known that each frame of video is recognized as an image. Therefore, each frame of collected videos is extracted into a folder, and the frames which contain instances of car, pedestrian, motorcyclists and bicycle cyclists are selected to compose a new folder for constructing the custom dataset. By considering the difficulty for a trained detection model to classify the difference between motor cyclists and bicycle cyclists is very challenging in the DVS condition, because the edge of event-based data may not be completely so that the precision of detection results can be lower. Hence, the motorcyclists and bicycle cyclists are classified into one class called riders. Finally, there are 1700 images extracted from collected traffic scene videos. Almost all target instances are included in each image to increase the training efficiency and avoid the dataset being trained too easily compared with the testing and validation sets. There are three sub-folders to divide 1100 images into a training set, 400 images into a testing set and 200 images into a validation set. All images contain the target objects from different angles to learn different features. However, the convolutional layers do not understand what the target objects are in the unlabeled images. Therefore, the next step is to annotate the collected dataset with the correct name labels to ensure the artificial neurons can learn the target objects rather than the unrelated components.

4.2.2 Dataset annotation

The Python manual labelling tool ‘LabelImg’ is installed under the Anaconda environments with the 3.9 Python version. This tool allows for drawing the bounding boxes and defining the class names manually so that the target objects are easily separated from background. It helps neurons to know which features are needed to learn and which features require to be recognized as the noise or background. A text file is needed to configure the annotating label names, the algorithm 3 is given below.

Algorithm 3: Define the target classes in the LabelImg’s text file

Define target classes
NC: 3
Classes
‘Car’
‘Rider’
‘Pedestrians’

The target object names are defined in this file. The labelling process is shown in Fig.30, the riders and cars are annotated with the same size of bounding boxes, and it is free to define which object is the target object. The point is that the different instances are annotated with different bounding boxes. For example, two cars are overlapping with parts of the entity, it is a fault to draw only one box to contain two cars. Two bounding boxes can still be drawn separately even though they overlap parts of the body. After annotating all instances, it is clear to find that the same class of instances are surrounded by the same color boxes.

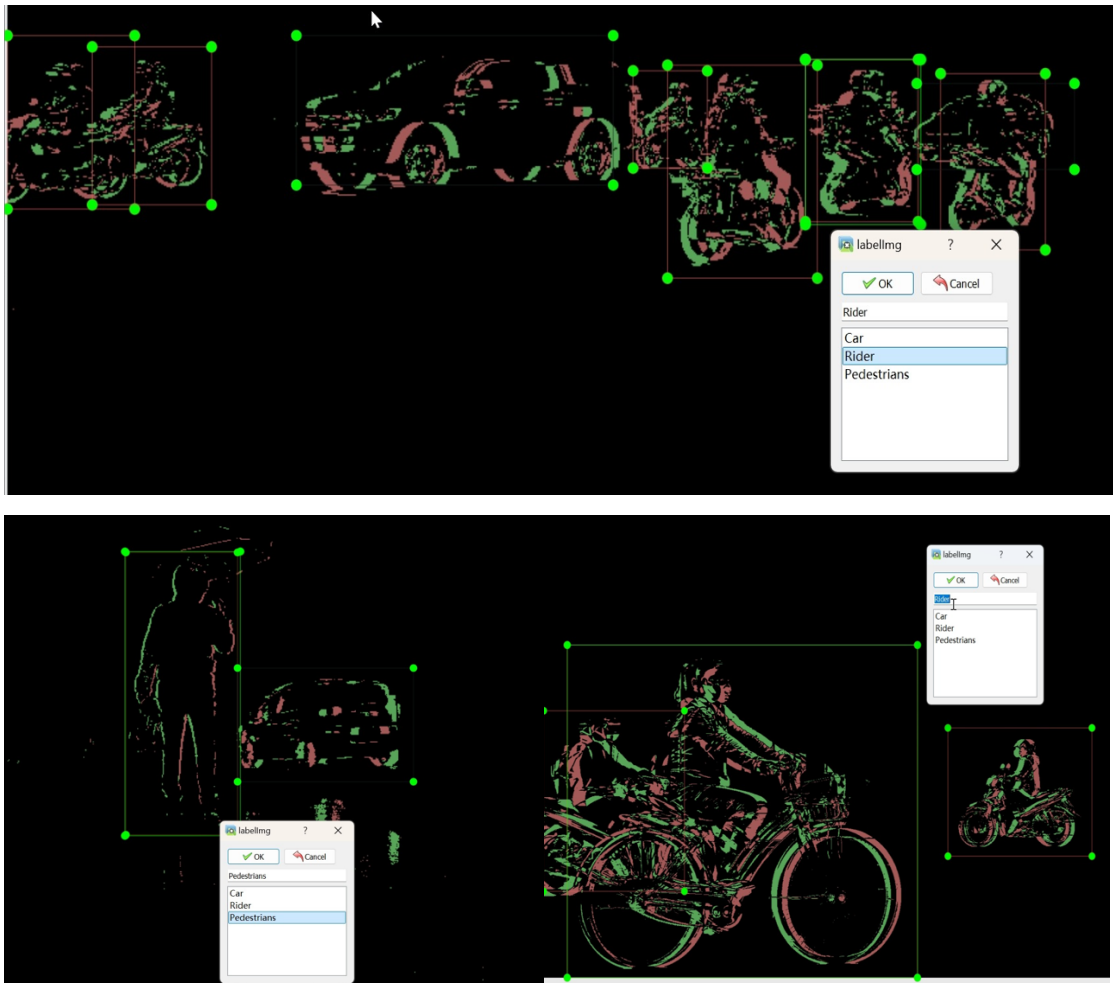


Figure 30: The annotation of DVS frames

The YOLO format text with the form of '<class_id> <x_center> <y_center> <width> <height>' can be output to save after labelling all instances in each frame. All coordinates are normalized with the value between 0 to 1, and each line in this text file represents one bounding box position.

4.3 Training of YOLO model

The YOLOv5 and YOLOv8 are trained on the Google Colab platform, which provides T4 GPU to train at a fast speed. The first step is to train the YOLOv5 detection model. The code to deploy the training process is shown in Algorithm 4, which contains three steps, including importing YOLOv5 by installing the ultralytics package, unzipping the well-organized custom dataset and deploying the training requirements with batch size and model version. In this experiment, the versions of nano, small, medium, large and extra-large for YOLOv5 are trained with the batch sizes of 16, 32, 64 for the epochs of 50, 75, 100 and 125, respectively. Based on the ability of the Yolo algorithm to continuously learn the input features and be able to change the pre-trained weights and biases, the detection model is trained on the basis of the pre-trained weights, which speeds up the training process and achieves higher accuracy with fewer training cycles. The training can be started successfully as the structure of dataset is readable by YOLO. The zip file of custom dataset is uploaded first before unzipping it. Algorithm 4 shows the process of training the weights of YOLOv5s based on a batch size of 16 for 100 epochs with the input images of 640×640 height and width. These parameters can be adjusted to optimize the training process based on the obtained training results.

Algorithm 4: The Python code to train the YOLOv5 detection model

```
# Clone GitHub repository, install dependencies and check Pytorch and GPU
!git clone https://github.com/ultralytics/yolov5 # clone
%cd yolov5
%pip install -qr requirements.txt comet_ml # install
import torch
import utils
display = utils.notebook_init() # checks
# Unzip custom dataset
!unzip -q ../dataset.zip -d ../
# Training configuration for the Ultralytics train.py
!python train.py --image 640 --batch 16 –epochs 100 --data custom.yaml --weights yolov5s.pt --cache
# Export the trained weights after finishing the training process
```

Algorithm 5: The Python code to train the YOLOv5 detection model

```
# Install dependencies
!pip install ultralytics
# Unzip custom dataset
!unzip -q ../dataset.zip -d ../
# Training configuration for the Ultralytics train.py
import os
from ultralytics import YOLO
model = YOLO("yolov8x.pt")
results = model.train(data=os.path.join("/custom_dataset", "/content/custom.yaml"), batch = 16, epochs = 125)
success = model.export(format= "tflite") # Export with the format of tflite
```

Algorithm 5 shows the training steps of YOLOv8, which is very similar to YOLOv5 since it is improved based on YOLOv5's framework. The export format is tflite, which runs under the TensorFlow Lite framework rather than remaining in PyTorch or other full-framework formats to align with the needs of resource-constrained edge computing environments, particularly for deployment in vehicles. The TFLite format offers significant advantages in terms of model size reduction, faster inference, and lower energy consumption, making it ideal for embedded systems and real-time applications. Compared to full-scale frameworks like PyTorch or TensorFlow, TFLite models are highly optimized and lightweight, which allows the trained detection model to run efficiently on devices with limited memory, compute power, and battery capacity, satisfying the requirement to deploy on the onboard automotive systems. This makes it possible to perform traffic participants detection tasks without relying on cloud computation, ensuring low latency and greater data privacy in autonomous or driver-assistance systems.

The configuration of yaml file represented in Algorithm 5 is needed to define the classes for the custom dataset and give the detailed paths of the training set, testing set and validation set, respectively. It helps YOLO to understand the meaning of numbers 1,2, and 3 represent which target class is in the YOLO format text file. This file is suitable for both YOLOv5 and YOLOv8.

Algorithm 6: The configuration of yaml file for YOLOv5 and YOLOv8.

```
# Configuration for custom YOLO dataset
Train: ../train_images_path/
Test: ../test_images_path/
```

Val: ../val_images_path/

NC: 3

Classes

Names: [‘Car’, ‘Rider’, ‘Pedestrians’]

4.4 Evaluation of YOLO models’ performance

In this chapter, the performances of trained models are evaluated by the technical parameters, which are precision, recall rate, mAP, and confusion matrix of each pre-defined class in the dataset.

Precision is a parameter to measure the proportion of predicted positive detections that are correct in the dataset. As an example, there are truly 128 rider instances in the validation set, the precision determines how many instances succeed in recognizing motorcycles with the correct labelled class name ‘Rider’. If 110 instances are detected as the rider, there are 20 instances predicted wrong as the label of ‘rider’. As a result, precision is calculated with the value of 110 / (110 + 20) = 84.6%. It does not care about how many motorcycles are not successfully detected during the validation process, it only focuses on how many of the predicted motorcycle images are correct. The mathematical expression is shown in Equation 12:

$$Precision = \frac{true\ positive}{true\ positive + false\ positive} \quad (12)$$

Recall rate measures the proportion of actual positive instances that were correctly detected by the detection model. For example, the trained model outputs the result of 100 instances with the label of ‘Rider’ correctly, but the ground truth of annotated ‘Rider’ is actually 128 instances. Therefore, the model misses the 28 false negative results, which means they are truly riders but not successfully detected by the model. The recall rate is equal to 100 / (100 + 28) = 78.1%. The mathematical expression is shown in Equation 13:

$$Recall\ rate = \frac{true\ positive}{ture\ positive + false\ negative} \quad (13)$$

The parameter of AP is defined to measure the average precision over a single class, which is computed as the area under the Precision-Recall (PR) curve. This curve is generated by plotting precision vs recall at different confidence thresholds from 0 to 1. The mAP is the mean of AP scores across all classes, where Equation 14 is given below.

$$mAP = \left(\frac{1}{N}\right) \times \sum_{i=1}^{N} AP_i \quad (14)$$

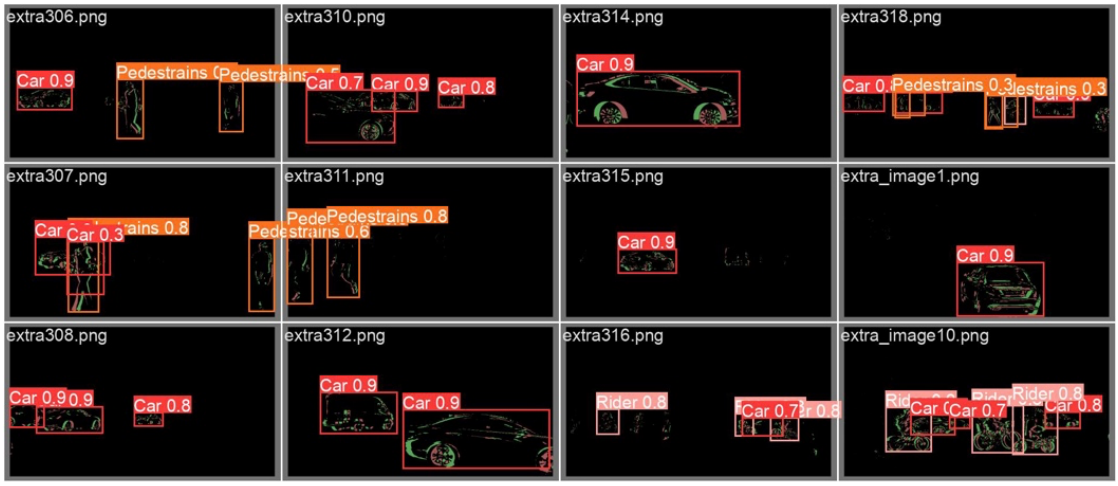
Where the N is the number of classes, AP_i is the AP for class i.

The F1 curve is a key graph to be used for identifying the optimal threshold value of the detection model. For detecting the pre-defined objects with a low noise level, a confidence score is defined as a minimum score to trigger the bounding box with the probability value and associated label for a frame. A low threshold value may lead the detection model to be sensitive to noise or background changes, which captures more unrelated objects. In contrast, a high threshold may filter out the important features that appear in a frame. If a pre-defined object exists in a frame, then the DVS may not capture it to trigger an event due to the brightness change at the corresponding pixel below the threshold, resulting in a missed detection. Equation 15 shows the mathematical expression of F1score.

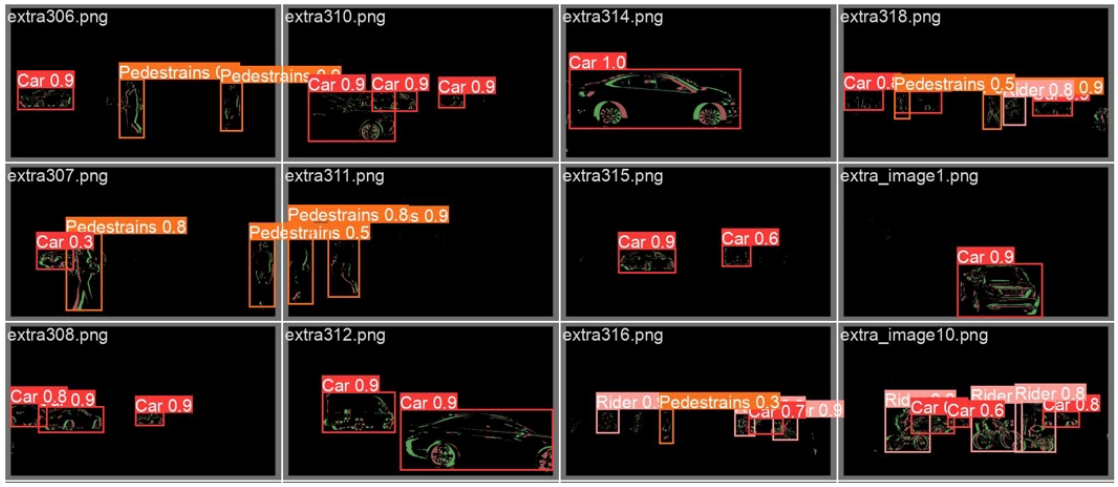
$$F1\ Score = 2 \times \left(\frac{precision \times recall\ rate}{precision + recall\ rate}\right) (15)$$

4.4.1 Validation results of trained YOLO model

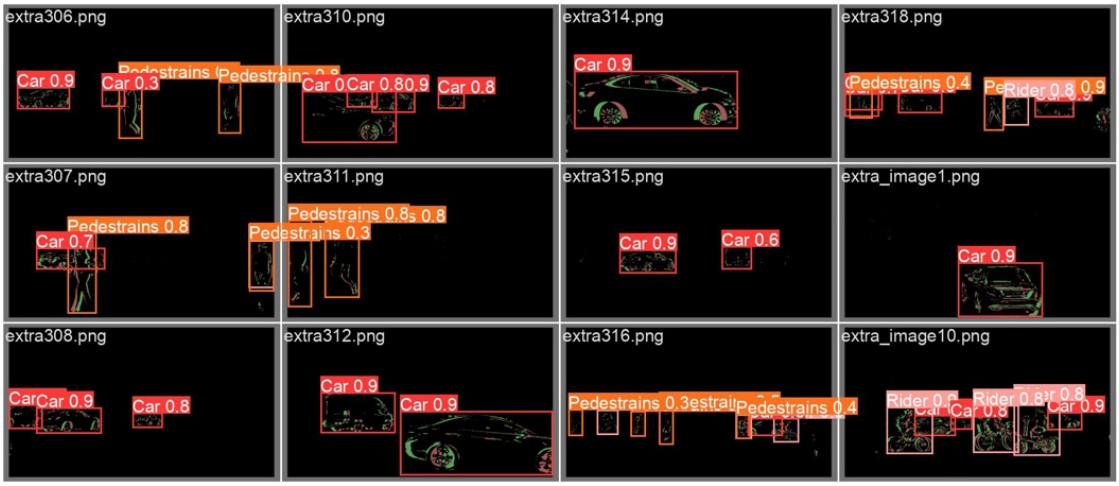
After finishing the training process of the YOLO model, the validation sets that contain the unlearned target data are used to evaluate the performance of the trained model by comparing the verified validation results with the ground truth. Each version of YOLOv5 is trained with the batch size of 16 for 100 epochs under the environment of GPU on CoLab by 7 hours totally, the results are shown in Table 3 and visualized in Fig.31. As can be seen clearly in Fig.31, the YOLOv5s has the better performance on predicting the targets than YOLOv5n, which is quite high precision to output the class probabilities and bounding boxes. YOLOv5m achieves a higher accuracy than the former two versions, because it avoids parts of wrong detection results and also increases the confidence score on the true results. It has a stronger probability of distinguishing the targets from the noise in background. Interestingly, the training result of YOLOv5l does not perform as well as YOLOv5m, since it has lower confidence scores on some targets. Obviously, the YOLOv5x has the best performance, it can filter the noise and output high confidence scores with the correct prediction. However, the visualized view is just given the subjective judgement. The evaluation matrices of these models are shown in Table 3, which presents the objective validation results.



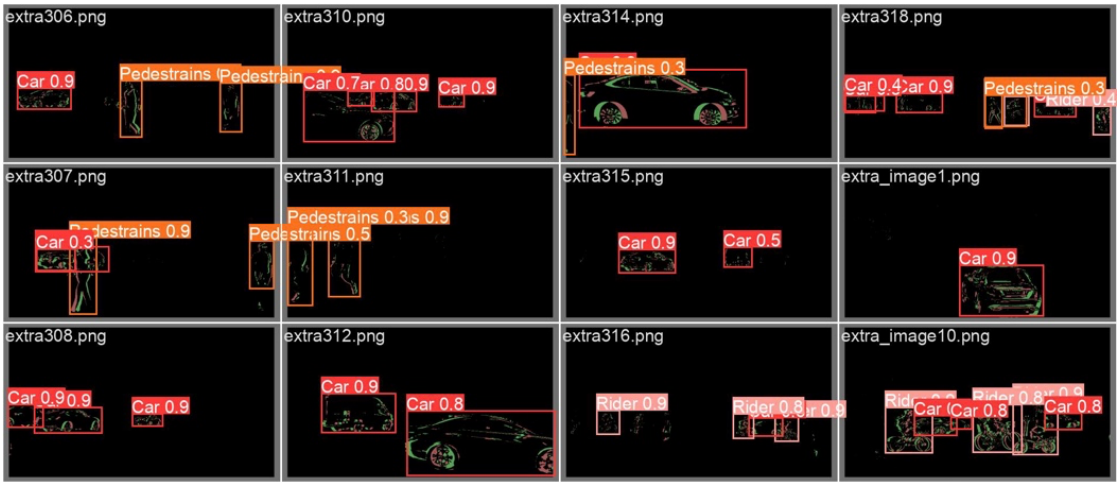
(a) YOLOv5n under the batch size of 16 with the epoch of 100



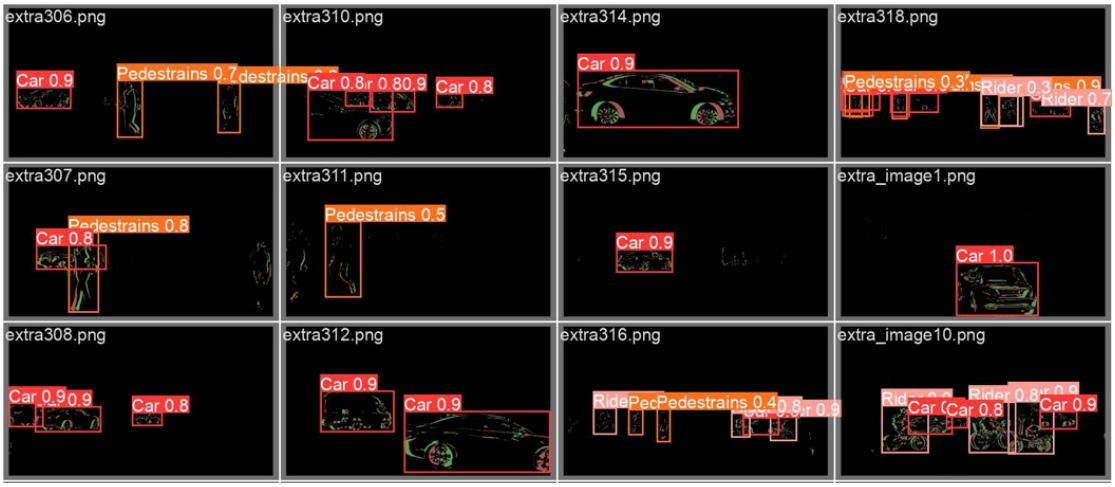
(b) YOLOv5s under the batch size of 16 with the epoch of 100



(c) YOLOv5m under the batch size of 16 with the epoch of 100



(d) YOLOv5l under the batch size of 16 with the epoch of 100



(e) YOLOv5x under the batch size of 16 with the epoch of 100

Figure 31: The visualized validation results for YOLOv5

The performance metrics collected during validation include precision, recall rate, mean average precision at 0.5 IoU (mAP@50), mean average precision across IoU thresholds from 0.5 to 0.95 (mAP@50_95), and the model weight sizes in megabytes. IoU (Intersection over Union) is a common metric used to evaluate how well the predicted bounding boxes match the ground truth boxes. The predicted box is considered a correct detection under a relaxed criterion if $IoU \geq 0.5$ while a strict criterion can be obtained if $IoU \geq 0.75$. These metrics provide analytical insights into each model's detection accuracy and deployment feasibility for real-time on-vehicle object detection using DVS.

As shown in Table 3, YOLOv5m achieves the highest mAP@50 score of 0.967, indicating its strong capability in detecting objects when evaluated with a relaxed IoU threshold of 0.5, meaning that even when the predicted bounding boxes do not perfectly align with the ground truth, they still overlap sufficiently at least 50% to be considered

correct, reflecting reliable object detection performance. The trained model of YOLOv5l improves the precision and recall rate from 0.893 and 0.924 to 0.903 and 0.934 compared with m version, respectively. However, the detection results of MAP are slightly lower than m version. It can be evidenced that the trained YOLOv5m based on this custom dataset has a stronger overall performance in terms of both classification and localization quality across a range of conditions than the versions of nano, small, medium and large. When it comes to extra-large version, which has the best prediction performances of mAP@50 and mAP@50-95 while maintaining the super high precision of 0.934 and recall rate of 0.91. However, YOLOv5x also occupies the largest size of 166MB, which is nearly four times larger than YOLOv5m.

YOLOv5s offers a highly practical compromise between model size and performance. With a weight size of just 13.6MB, it yields a strong mAP@50–95 of 0.649, nearly matching the accuracy of larger models. Although its precision is slightly lower at 0.897, this is still acceptable to conduct the real-time traffic monitoring on the edge computing devices, where detection speed and resource efficiency are more critical than complete object coverage. The YOLOv5n also has a good result by considering its extremely small model size and the fastest inference time.

Table 3: The validation results for each YOLOv5 version model based on the batch size of 16 with epochs of 100

Model version	Precision (all)	Recall rate (all)	MAP@50 (all)	MAP@50-95 (all)	Weights size
YOLOv5n	0.899	0.911	0.962	0.646	3.64MB
YOLOv5s	0.897	0.913	0.961	0.649	13.6MB
YOLOv5m	0.893	0.924	0.967	0.655	40.1MB
YOLOv5l	0.903	0.934	0.961	0.652	88.4MB
YOLOv5x	0.934	0.91	0.957	0.656	166MB

The impacts of changes in batch size and epoch on the training results of the model are also analyzed through the control variable method. In this experiment, the YOLOv5n is selected to train for observing the changes in results by various batch sizes and epochs since it takes up the shortest time and consumes the lowest energy. The experiment results are shown in Table 4, the models of YOLOv5n are trained under the batch sizes of 16, 32 and 64 with the epochs of 50, 75, 100 and 125, respectively.

At the smallest batch size of 16, the model performs excellently across various epochs. Notably, it achieves the highest precision of 0.943 at 125 epochs, indicating high accuracy in classifying positive detections. However, its recall rate drops to 0.894, which

is slightly lower than the peak of 0.911 at 100 epochs, meaning that there are some missed true detections for the targets at larger epochs. This study finds that a small batch size paired with longer training cycles improves the model's precision, although it may be dependent on the quality and quantity of training and validating datasets.

Training with a batch size of 32 shows a relatively consistent performance with slightly more fluctuation. The best recall rate of 0.905 is achieved at 100 epochs, while precision remains close to 0.9 across all configurations. The parameters of mAP@50–95 are all around 0.640, which are slightly below the results under the batch size of 16. Overall, the predicted performance doesn't significantly improve with increased epochs beyond 100. At 125 epochs, the recall rate obviously drops. This could indicate excessive training to lead the over-fitting on this batch size.

With a larger batch size of 64, the model achieves consistently high precision at 0.92 to 0.938, showing strong confidence in predictions. However, recall rates become lower with a minimum of 0.871 at 75 epochs, which means the model tends to miss more true positives as the batch size increases. Even though the mAP@50 ranges between 0.947 and 0.962, it does not outperform the smaller batch sizes. This could be due to larger batches reducing the variance in weight updates, making the model converge faster but less adaptively.

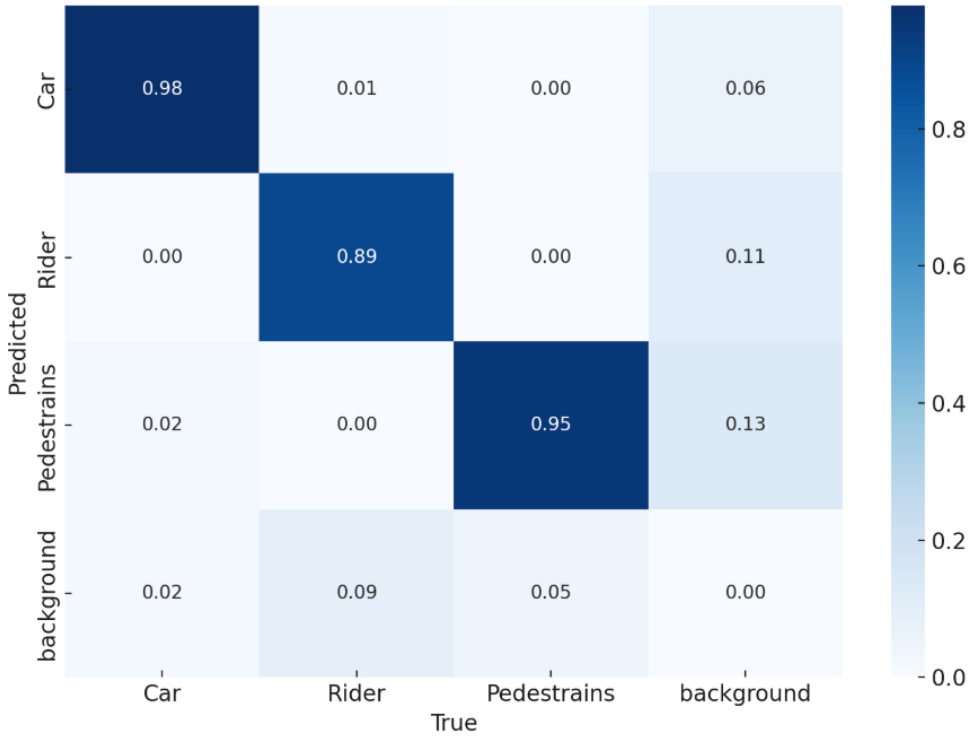
Table 4: The validation results for each YOLOv5n based on batch sizes of 16, 32 and 64 with epochs of 50, 75, 100 and 125

Batch size	Epoch	Precision (all)	Recall rate (all)	MAP 50 (all)	MAP 50-95 (all)
16	50	0.903	0.903	0.96	0.648
	75	0.902	0.904	0.958	0.638
	100	0.899	0.911	0.962	0.646
	125	0.943	0.894	0.966	0.643
32	50	0.903	0.898	0.962	0.640
	75	0.893	0.878	0.952	0.639
	100	0.899	0.905	0.958	0.640
	125	0.900	0.874	0.954	0.643
64	50	0.923	0.884	0.962	0.638
	75	0.938	0.871	0.956	0.642
	100	0.923	0.882	0.947	0.642
	125	0.92	0.895	0.958	0.640

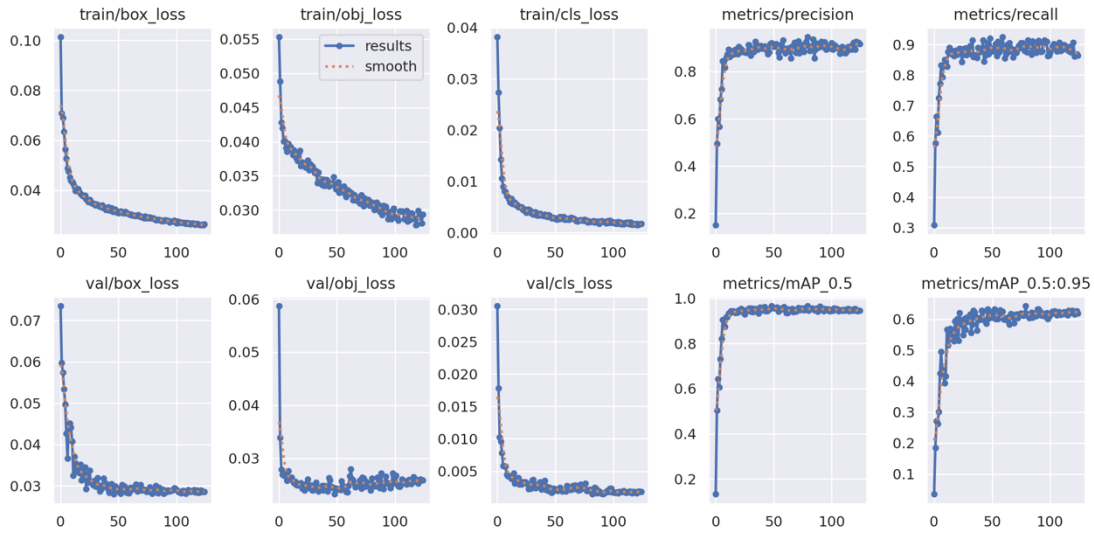
Obviously, the YOLOv5n with 16 batch size under 125 epoch has the best performance. Fig.32(a) shows the validation results for each trained class, the precisions of car, rider and pedestrian are all above 0.90. But the parameters of recall rate are slightly lower compared with precision. There seems to be a trade-off between these two parameters, the target class 'rider' has the highest precision with 0.99, while its recall rate is the lowest at 0.841. In the next step to deploy the trained model into the main program, it is necessary to define the detection threshold based on the F1 curve rather than setting the value at will. Fig.32(b) shows that the classification performance of the trained YOLOv5n model across four categories, including Car, Rider, Pedestrians, and Background. The model shows strong accuracy in detecting Cars (0.98) and Pedestrians (0.95), with very few misclassifications. Riders are predicted correctly 89% of the time, but there is some confusion, particularly with Pedestrians and Background. The Background class suffers the most misclassification, 13% of Pedestrians are incorrectly classified as background, indicating the model sometimes fails to detect objects in cluttered scenes or under low-contrast conditions. Overall, the model performs well on Cars and Pedestrians but still struggles slightly with distinguishing between Riders and Background. Fig.32(c) shows the training results of the best trained model for YOLOv5n under 125 epochs with a batch size of 16. There are three points to explain why it is just fitting to give an excellent performance. Firstly, both training and validation box loss consistently decrease over the epochs, starting from around 0.10 (train) and 0.07 (validation) and levelling off around 0.025 to 0.03. This indicates that the model has learned to localize objects more accurately over time, and the convergence suggests no signs of overfitting or underfitting. Secondly, the objectness loss indicates how well the model distinguishes object presence, it also shows a steady decline dropping below 0.03 for both training and validation, which reflects reliable learning in detecting object regions. Thirdly, classification loss starts around 0.04 and quickly converges near 0.005. The low and steady values on both train and validation sets evidence effective learning of class labels with minimal confusion between classes. What is more, the curves of precision, mAP_50 and mAP_0.5:0.95 keep the increasing trend until reaching their peak points. The recall rate is slightly dropping after training 110 epochs, which is also acceptable.

Class	Images	Instances	P	R	mAP50	mAP50-95
all	200	618	0.943	0.894	0.966	0.643
Car	200	194	0.929	0.949	0.977	0.688
Rider	200	126	0.99	0.841	0.967	0.629
Pedestrains	200	298	0.911	0.893	0.955	0.611

(a) Validation results



(b) Confusion Matrix



(c) The performance matrices of training results

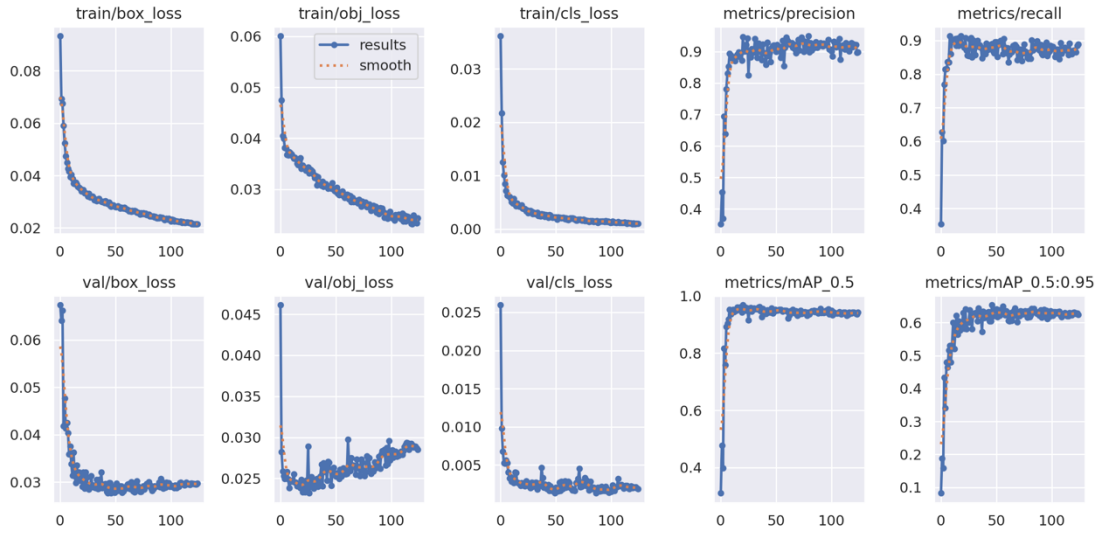
Figure 32: The Training results of YOLOv5n under 125 epochs with a batch size of 16.

Based on the experimental results above, it was found that the training performance is better under the epoch of 125 with a batch size of 16 for this custom dataset. Comparison of all trained YOLOv5 models under the epoch of 100, the s version has the best performance despite the x version. Therefore, the next step is to train YOLOv5s under an epoch of 125 with 16 batch size to compare the benchmark results of YOLOv5n. As can be seen in Figure 33, the training results of YOLOv5s are unpredictably worse

than YOLOv5n by deploying the same configurations. The reason is that it is over-fitting since the validation box loss has risen sharply after passing the bottom point from 0.025 to 0.30. Its recall rate and precision are all presented with a downward trend after training 100 epochs.

Class	Images	Instances	P	R	mAP50	mAP50-95
all	200	618	0.913	0.89	0.957	0.654
Car	200	194	0.867	0.954	0.976	0.71
Rider	200	126	0.945	0.857	0.947	0.643
Pedestrains	200	298	0.927	0.859	0.948	0.608

(a) Validation results

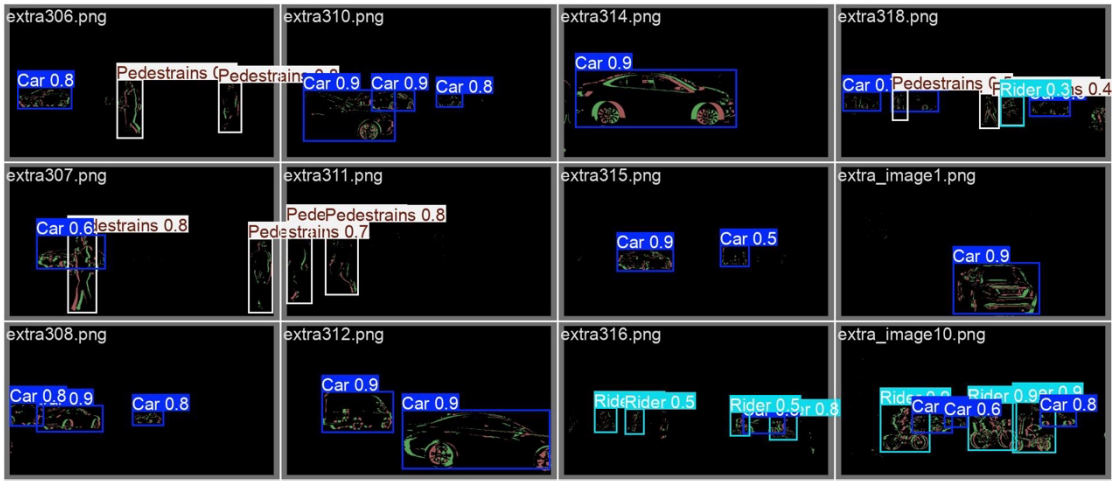


(b) The performance matrices of training results

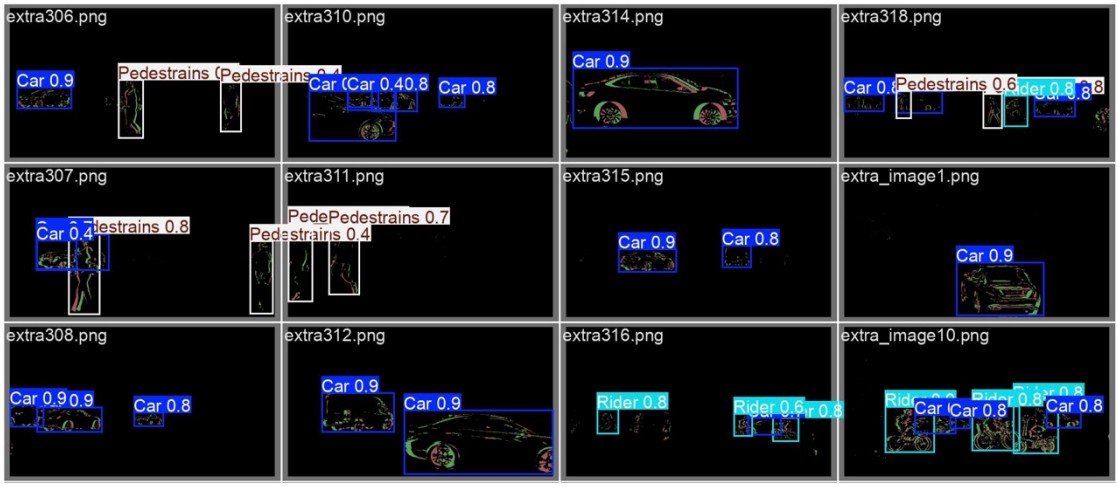
Figure 33: The training results of YOLOv5s

During the training step, the model of YOLOv5n under the epoch of 150 with the batch size of 16 also trained, but the parameters of precision and recall rate drops obviously compared with the results from epoch of 125 since the curve of training loss and validation loss rises after experiencing the bottom point, indicating that it is over-fitting by excessive training cycles.

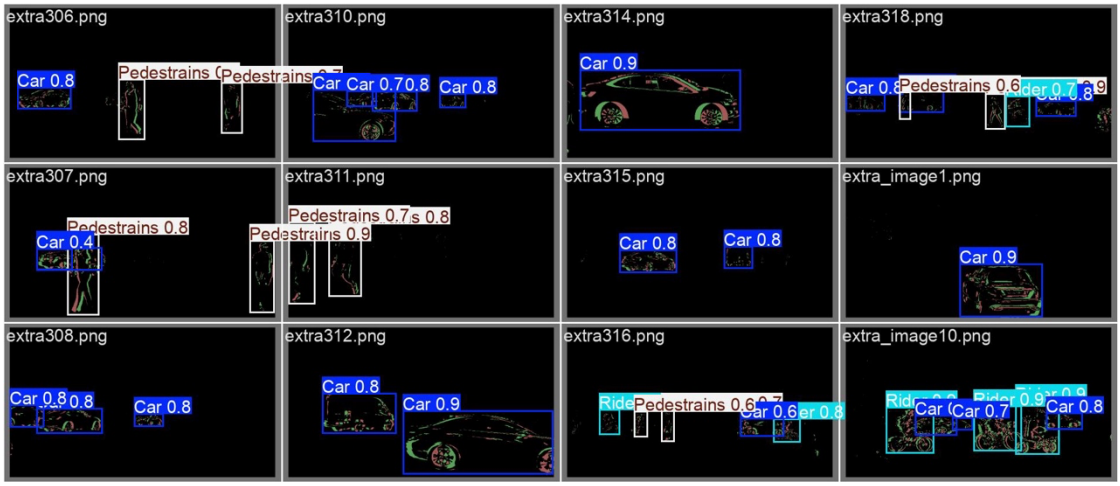
The visualized YOLOv8 validation results are shown in Fig.34, by comparing with YOLOv5, YOLOv8n predicts the higher class probabilities for cars, especially for the condition of multi-class overlapping. There is no significant improvement in YOLOv8s, since both models' s-version are missed true results. In YOLOv8m, it has a poorer detection performance on overlapped objects, but it can detect the single target by outputting higher confidence score. YOLOv8l has the obvious improvement to detect in all kinds of conditions, it has a stronger ability to recognize more unclear features from the dark background. However, YOLOv8x has worsen performances compared with YOLOv5x, it cannot classify the targets from the noise conditions.



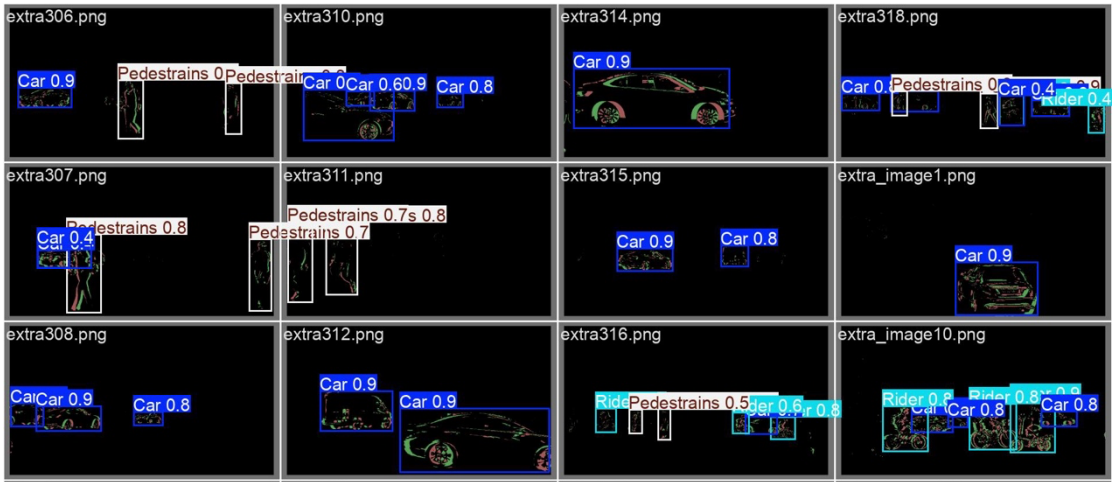
(a) YOLOv8n under the batch size of 16 with the epoch of 100



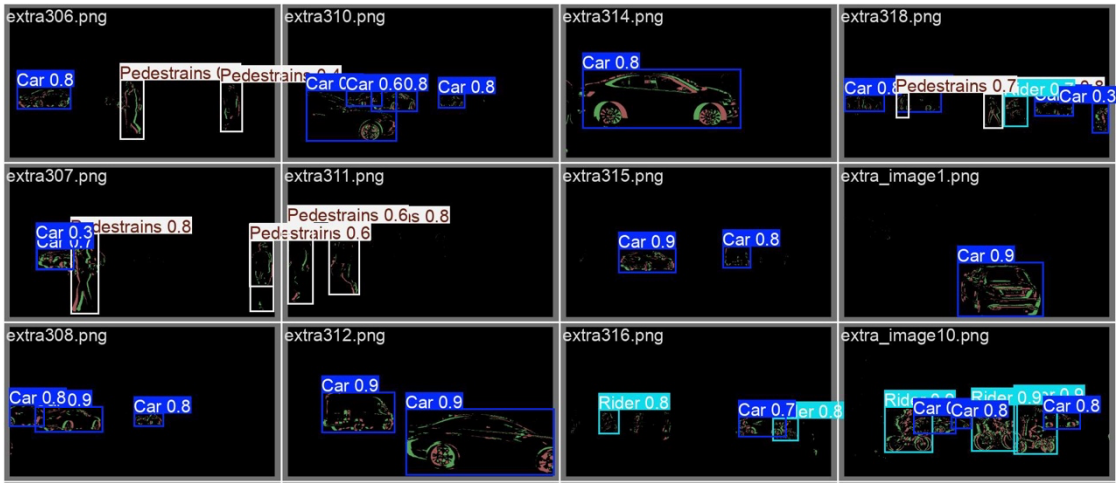
(b) YOLOv8s under the batch size of 16 with the epoch of 100



(c) YOLOv8m under the batch size of 16 with the epoch of 100



(d) YOLOv8l under the batch size of 16 with the epoch of 100



(e) YOLOv8x under the batch size of 16 with the epoch of 100

Figure 34: The visualized validation results for YOLOv8

The training results of YOLOv8 are shown in Table 5, the most obvious difference compared with YOLOv5 is that the precision parameters in medium and large versions are improved from 0.893 and 0.903 to 0.922 and 0.921. The mAP values are slightly lower than those of YOLOv5. However, the weight sizes for small versions like nano and small are approximately increased by 1.5 times, while the extra-large version has surprisingly decreased its size from 166MB to 130.6MB. Overall, the performances in various YOLOv8 versions are not better than YOLOv5, obviously.

Table 5: The validation results for each YOLOv8 version model based on the batch size of 16 with epochs of 100

Model version	Precision (all)	Recall rate (all)	mAP@50 (all)	mAP@50-95 (all)	Weights size
YOLOv8n	0.937	0.872	0.95	0.669	5.94MB
YOLOv8s	0.903	0.865	0.949	0.653	21.4MB

YOLOv8m	0.909	0.903	0.953	0.655	41.7MB
YOLOv8l	0.922	0.891	0.953	0.661	83.73MB
YOLOv8x	0.921	0.892	0.949	0.65	130.6MB

As shown in Fig.35, YOLOv8n employs a unified head with Distribution-Focal Loss (DFL), causing substantially larger loss magnitudes and a gradual decay over the full cycles. In contrast, YOLOv5n's three separate losses of box, objectness, and classification initialize at a much smaller scale from 0.04 to 0.10, then converge to near zero by epoch 40 to 50. The slower descent of YOLOv8n losses reflects both the numerical scale of DFL and its denser regression targets, whereas YOLOv5n's losses benefit from simpler targets and earlier saturation. Both models rapidly achieve high precision and recall rates. YOLOv5n reaches a recall rate above 0.90 and a precision above 0.85 by epoch 20, then keeps a stable trend to reach the highest point. YOLOv8n climbs more steadily, attaining similar values only around epoch 40, then maintains a marginally tighter band of validation fluctuations, indicating stronger generalization stability after convergence.

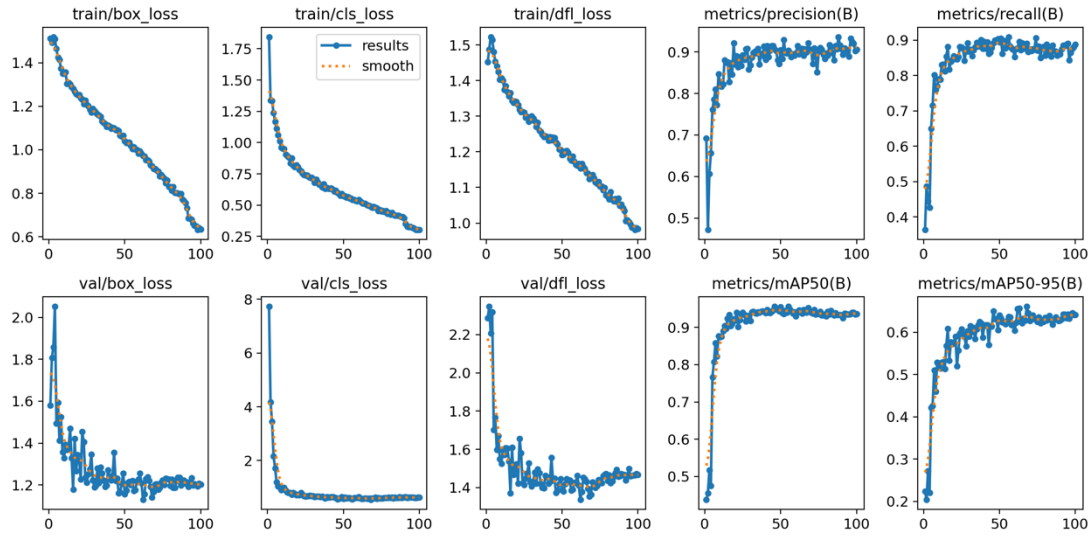


Figure 35: Training results of YOLOv8n

In conclusion, the YOLOv5x has the best performance on predicting the bounding boxes and outputting the class probabilities, but it is not suitable for deploy in this paper for the goal of real-time detection with low latency. YOLOv5n brings the surprising results under the environment of 16 batch size with 125 epochs, which achieves the highest precision and nearly 90% recall rate without over-fitting. YOLOv8 has better validation results when the larger size models are compared, indicating that there are some improvements on the architecture when training with more complex data based on YOLOv5. The larger batch size leads the lower detection performance since the faster

convergence cannot learn features and patterns well. As the epochs increase, the phenomenon of over-fitting can cause to lower in the validation results.

4.4.2 Detection results by integrating into the main loop

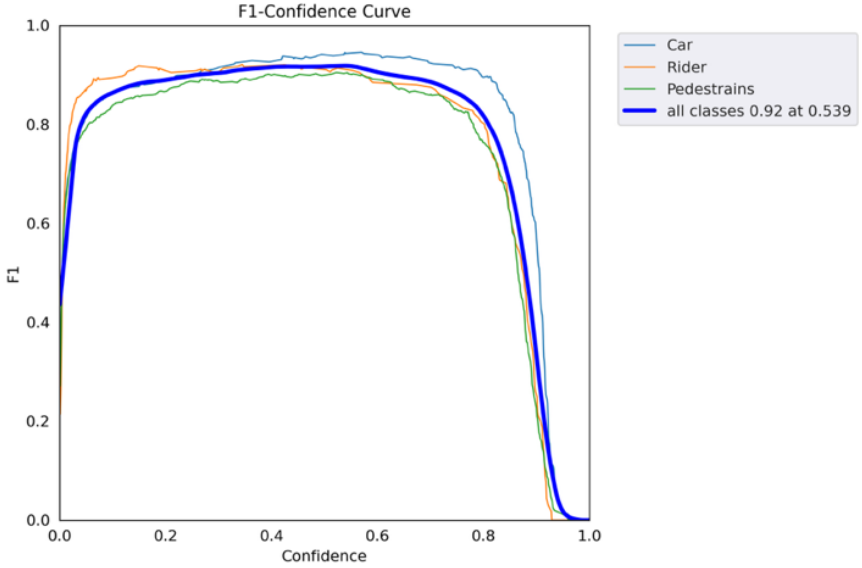
After training the YOLO models and evaluating each version's performance, the models are integrated into the main program, which consists of the preprocessing part. The integration of the overall system defines the input for each frame with 640×640 to match the input of YOLO. Then the frequency-based encoding and the function of blending are applied. Then, inferring the YOLO model under the TensorFlow Lite framework, which consumes less resources to realize the faster detection speed, the process of inferring YOLOv5n is shown in Algorithm 7. The class map is defined with the target labels and inference the model running on the RGB video to able to output bounding boxes, class names and confidence scores.

Algorithm 7: The Python code to infer YOLOv5 based on TFLite

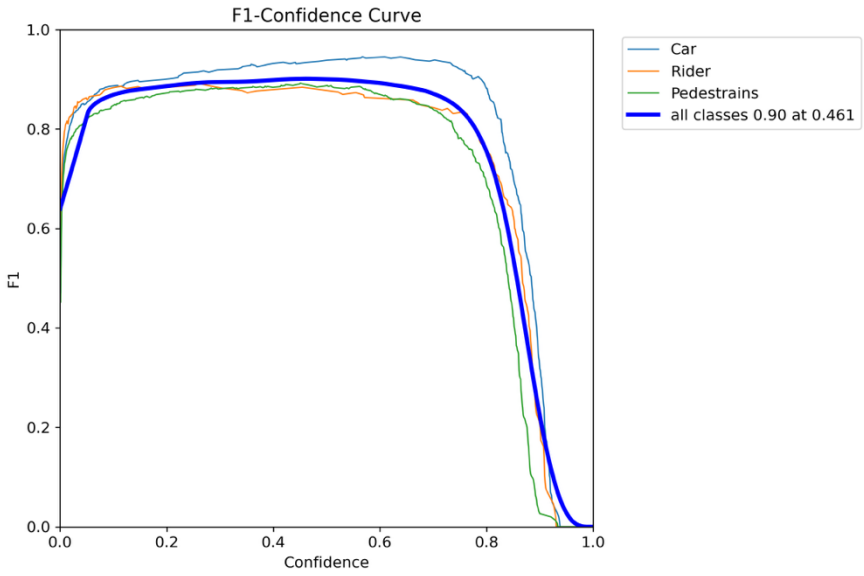
```
import tensorflow as tf
from tensorflow.lite.python.interpreter import Interpreter
# === Paths & settings ===
TFLITE_PATH = "/Users/huanghaiyang/Desktop/yolov5s.tflite"
YOLO_THRESHOLD      = 0.7
INPUT_SIZE              = 640
# === Class maps ===
class_idx2name = {0:"Car", 1:"Rider", 2:"Pedestrains"}
# === TFLite setup ===
interpreter = Interpreter(model_path=TFLITE_PATH, num_threads=NUM_THREADS)
interpreter.allocate_tensors()
i_idx = interpreter.get_input_details()[0]['index']
o_idx = interpreter.get_output_details()[0]['index']
# === YOLO inference on BGR image ===
def run_yolo(img_bgr):
    img_p, r, (dw,dh) = letterbox(img_bgr)
    inp = np.expand_dims(img_p.astype(np.float32)/255.0, 0)
    interpreter.set_tensor(i_idx, inp)
    interpreter.invoke()
    output = interpreter.get_tensor(o_idx)[0]
    boxes, scores, classes = [], [], []
    for det in output:
```

```
            conf = float(det[4])
            if conf < YOLO_THRESHOLD: continue
            x,y,bw,bh = det[:4]*INPUT_SIZE
            x = (x-dw)/r; y = (y-dh)/r
            bw/=r; bh/=r
            x1,y1 = int(x-bw/2), int(y-bh/2)
            x2,y2 = int(x+bw/2), int(y+bh/2)
            boxes.append((x1,y1,x2,y2))
            scores.append(conf)
            classes.append(int(np.argmax(det[5:])))
    dets=[]
    if boxes:
            keep = tf.image.non_max_suppression(
                    boxes, scores,
                    max_output_size=len(boxes),
                    iou_threshold=0.5,
                    score_threshold=YOLO_THRESHOLD
            ).numpy()
            for i in keep:
                    dets.append((boxes[i], scores[i], classes[i]))
    return dets
```

The YOLO threshold is set based on the F1 curve, which is shown in Fig.36. It shows that the highest precision of all classes at 0.92 when setting the confidence score to 0.539. Selecting the confidence score of 0.539 indicates that the detection model only outputs the prediction results when the detected target object's class probability is at least 53.9%. However, the confidence value of 0.539 is too low to cause a high false detection rate in backgrounds or unrelated components. There are many moving objects on the traffic scenes, so the confidence score at 0.539 can easily output the wrong detection results, and therefore, pre-setting its value at 0.7 is reasonable since there does not appear to be a dramatically decreasing trend of detection accuracy for changing the confidence score from 0.539 to 0.7. Similarly, YOLOv8n also maintains the high accuracy at the x-axis of 0.7, even though giving up the suggested confidence score of 0.461.



(a): F1 curve of YOLOv5n



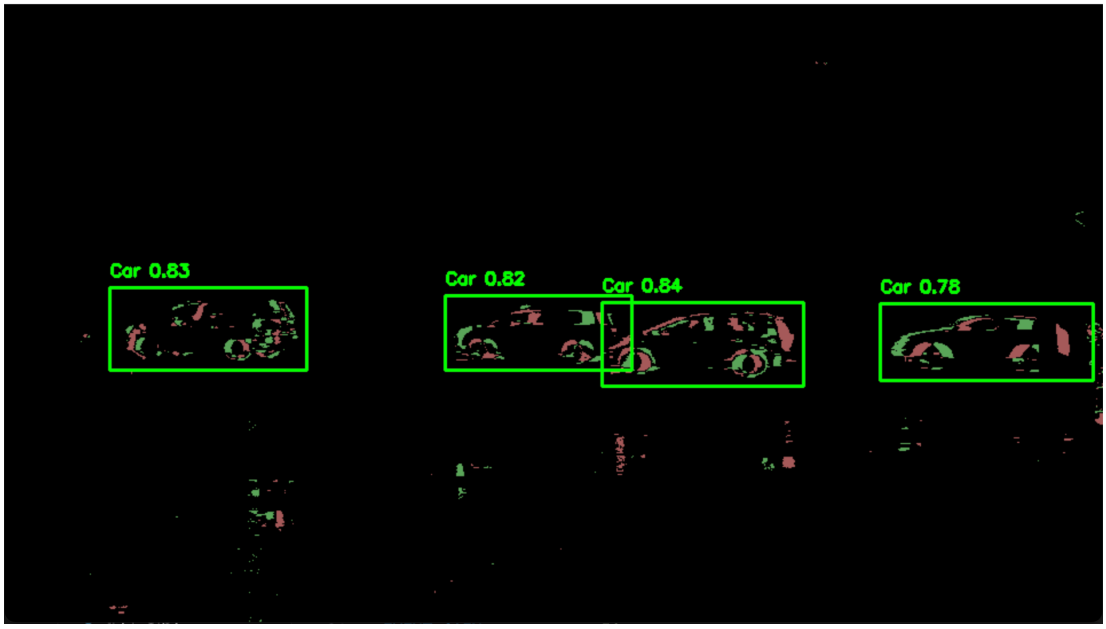
(b) F1 curve of YOLOv8n

Figure 36: The F1 curve of YOLOv5n and YOLOv8n

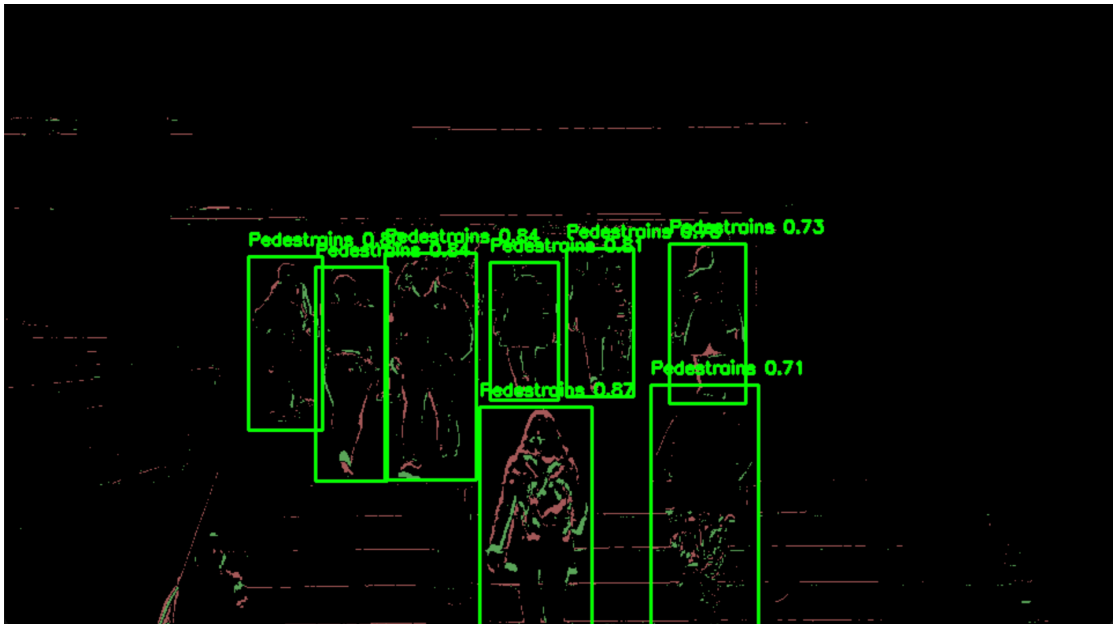
The overall designed system runs on two different kinds of hardware, including PCs with Apple M1 chip and NVIDIA RTX 4060. The detection results of the main program design in this paper based on YOLOv5n are shown in Fig.37, and the comparison results of inference time and FPS between two devices are shown in Table 6.

In Fig.37(a), the testing scene is under the raining condition, leading the moving instances' shallow cast onto the puddle on the ground to increase the noise within background. However, all four car instances can be detected successfully with a high confidence score from 0.78 to 0.84. Fig.37(b) shows a monitoring of the scene of the

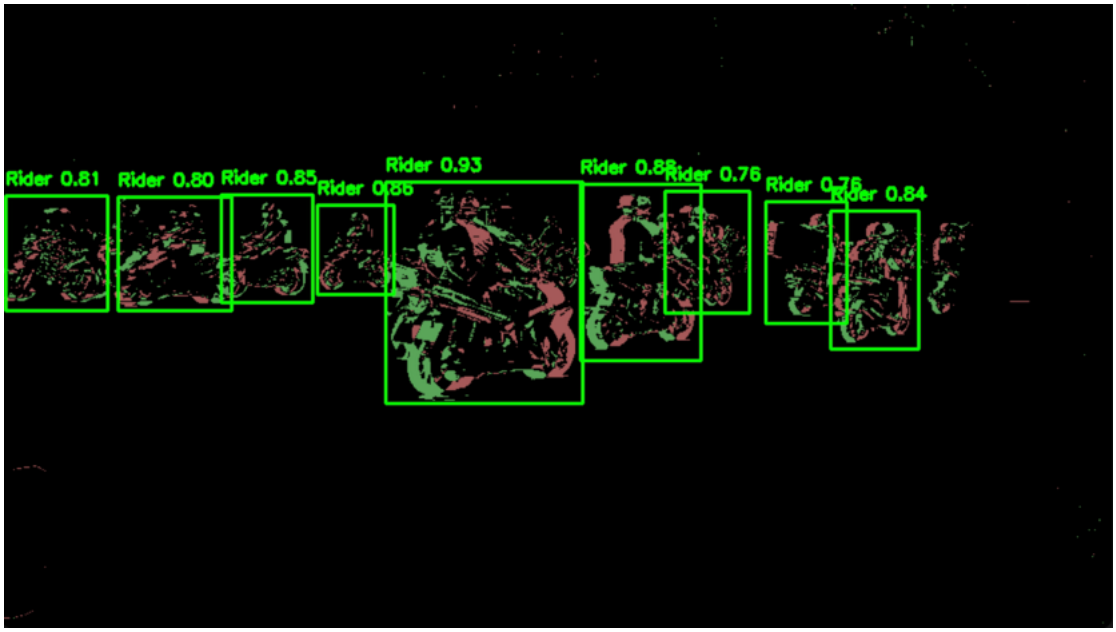
staircase entrance of the pedestrian overpass, even though the static camera seems slightly swung by the wind to add noise, all pedestrians can be detected successfully, and the model is able to output the bounding boxes to cover the entire body of the instance. Fig.37(c) captures the scene of a bustling road, and each rider is detected with high precision from the range of 0.76 to 0.93. The application of non-maximum suppression ensures there is no overlapping bounding boxes appear on the same instance.



(a) Detection results of car instances



(b) Detection results of pedestrian instances



(c) Detection results of rider instances

Figure 37: Detection results with YOLOv5n

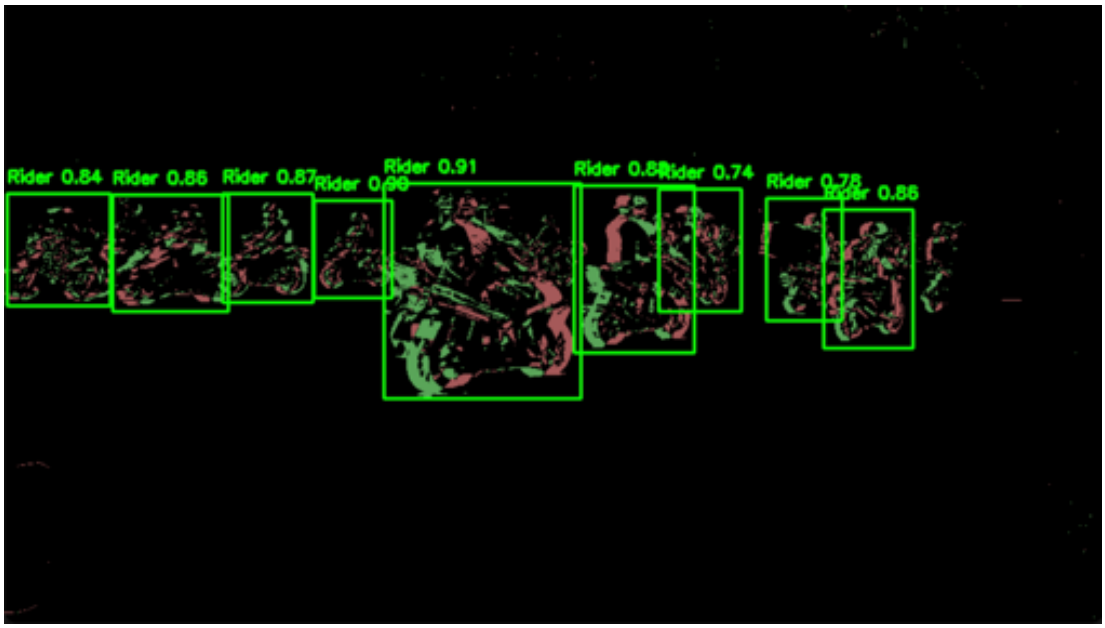
Table 6: The FPS performance on two hardware platforms based on YOLOv5n

FPS	Apple M1 (CPU)	NVIDIA RTX 4060 (CPU)	NVIDIA RTX 4060 (GPU)
0-10 Frame	8.84	25.35	62.57
11-20 Frame	10.77	30.66	66.59
21-30 Frame	13.82	38.76	62.49
31-40 Frame	14.37	39.79	67.35
41-50 Frame	13.97	39.01	78.99
51-60 Frame	14.00	38.89	60.56
61-70 Frame	13.87	38.39	62.19
71-80 Frame	13.85	38.39	74.01
81-90 Frame	13.68	38.17	77.01
91-100 Frame	13.77	37.84	76.84
101-110 Frame	14.24	38.96	79.95
111-120 Frame	13.47	37.87	71.44
121-130 Frame	13.42	37.57	78.57
131-140 Frame	13.04	36.55	76.85
Average Inference time (Seconds)	0.08	0.03	0.01

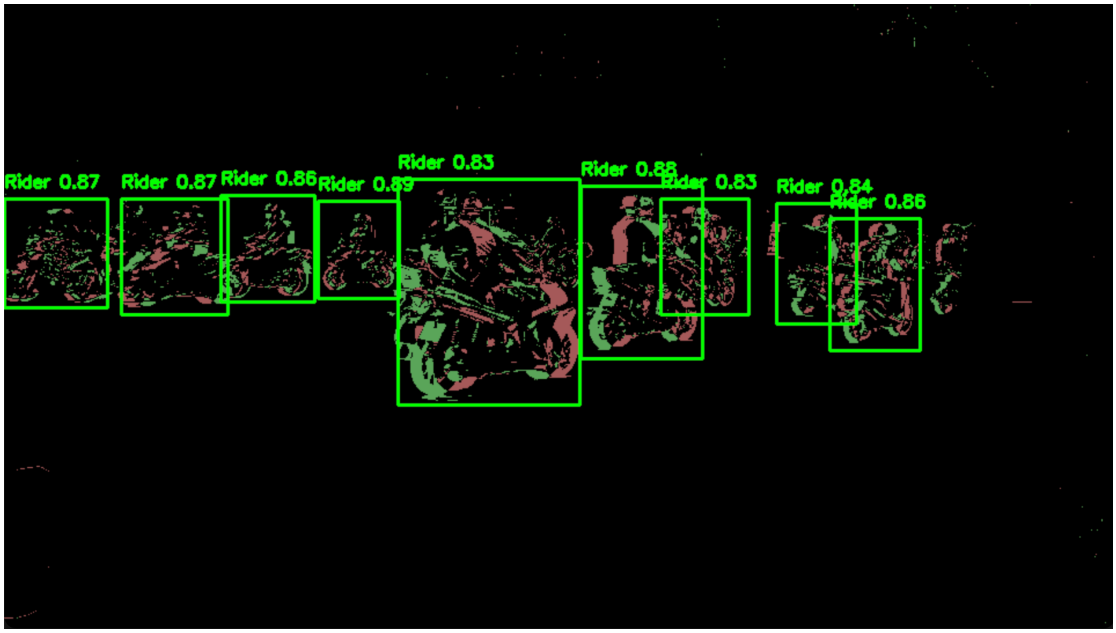
As the records of FPS for every 10 detected frames and the average inference time in Table 6, running YOLOv5n model on Apple M1 based on CPU has the worst

performance with the FPS range from 8.84 to 14.37. Even though the multi-threads are deployed to allow the TensorFlow Lite framework to use the 8 CPU threads in parallel for model inference, the results do not achieve the expectation to achieve high FPS. On the contrary, the program has a significant improvement when running on the Windows PC of NVIDIA RTX 4060, the performance under the CPU can reach the highest point at 39.01, which is also acceptable for real-time detection when dropping to the lowest point at 25.35. The designed system under NVIDIA RTX 4060 achieves the ability to perform quick inference of model, whose inference time is multiple times shorter than that in Apple M1. The experiments of exploring GPU on Windows are also conducted successfully by exporting the model with ONNX format rather than tflite, because TensorFlow Lite cannot support activating GPU running the YOLO model. Even though trying to deploy ONNX on Mac, it fails since the Apple M1 does not delegate GPU acceleration or accept the format of ONNX. What is more, the Apple Silicon does not support CUDA, which is the most mature and widely used GPU backend for AI frameworks. As a result, the FPS improves nearly 2 times to 79.95 at peak, and it also speeds up the inference time to 0.01 seconds on wins. The code of implementation is shown in the Appendix.

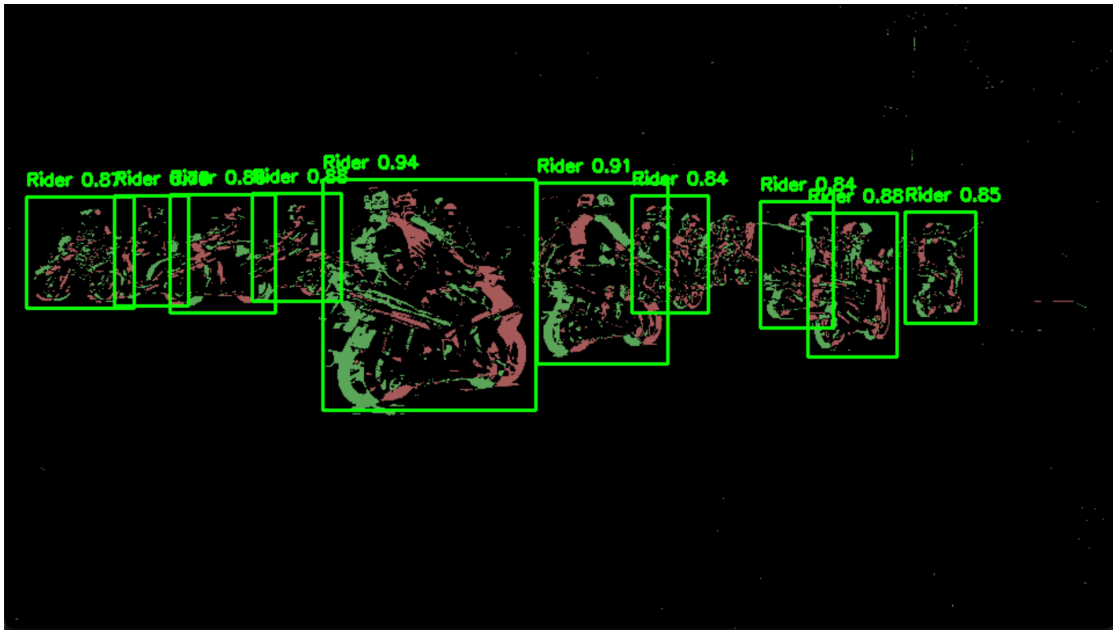
What is more, despite the YOLOv5n that is shown in Fig.37, the deployed system performances for versions of YOLOv5 are also shown in Figure 38. Even though captured at the same frame, the lower FPS of the larger version still lead to the inevitable latency. Therefore, the frames in Fig.38 have slight differences from each other. The detection results from the deployment of each model to detect the target objects in the same traffic scene demonstrate that the larger version of the YOLO model has the better performance to output the higher confidence score on target objects even though the training results show that the nano version and small version has the higher detection accuracy than medium and large version. However, the nano version has still retained a strong ability to be deployed for conducting the real-time detection tasks. The YOLOv8 performs nearly the same results as YOLOv5, all versions' results are summarized in Table 7.



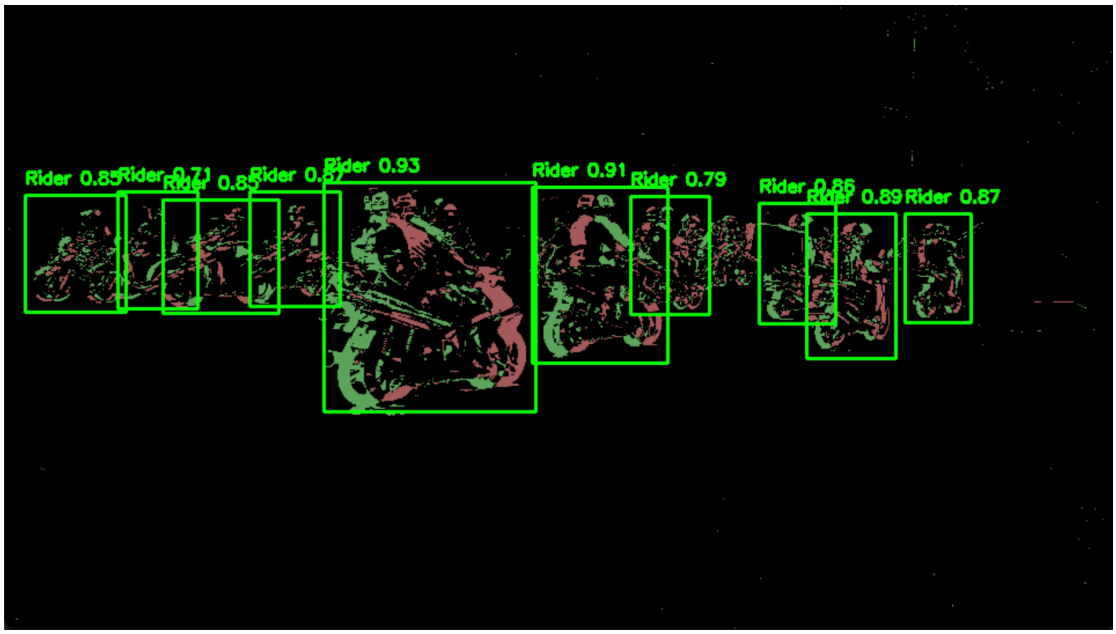
(a) Detection results based on YOLOv5s



(b) Detection results based on YOLOv5m



(c) Detection results based on YOLOv5l



(d) Detection results based on YOLOv5x

Figure 38: Detection results of YOLOv5

To be observed in Table 7, running under the environment of Apple M1 leads the poor FPS performances, especially for large versions. As the latency increases to 200ms, the real-time detection fails to deploy, and the designed system can only be run asynchronously with the FPS that is lower than 5. Even though the tflite framework is deployed to save the resource cost, the FPS performance is not acceptable to do real-time traffic monitoring.

Table 7: The performances of the YOLO model under 125 epochs with 16 batch size based on Apple M1 CPU

Model	Average FPS (every frame)	Average latency (every frame)	Model size (tflite)
YOLOv5n	13.22	75.64ms	3.6MB
YOLOv5s	6.87	145.56ms	14.2MB
YOLOv5m	2.90	344.83ms	41.9MB
YOLOv5l	1.41	709.22ms	92.4MB
YOLOv5x	0.79	1265.82ms	172.6MB
YOLOv8n	13.79	92.68ms	12.3MB
YOLOv8s	7.13	140.25ms	44.8MB
YOLOv8m	3.92	255.10ms	103.7MB
YOLOv8l	1.45	689.67ms	175.0MB
YOLOv8x	0.78	1282.05ms	273.1MB

In Table 8, each model's performance is evaluated based on the same frame within one test traffic scene's 120s video, and the multi-threads are also applied to speed up the inference speed in parallel with 8 lines to improve FPS as much as possible. The FPS performance of each version improves significantly, YOLOv8x can detect in video with nearly 20 FPS, even though it consumes too much energy. Overall, two models with various versions all have good performances under GPU operation, which not only realize the fast speed to inference the YOLO model, but also provide high average precision on detecting cars, riders and pedestrians.

Table 8: The performances of YOLO model under 125 epochs with 16 batch size based on NVIDIA RTX 4060 GPU

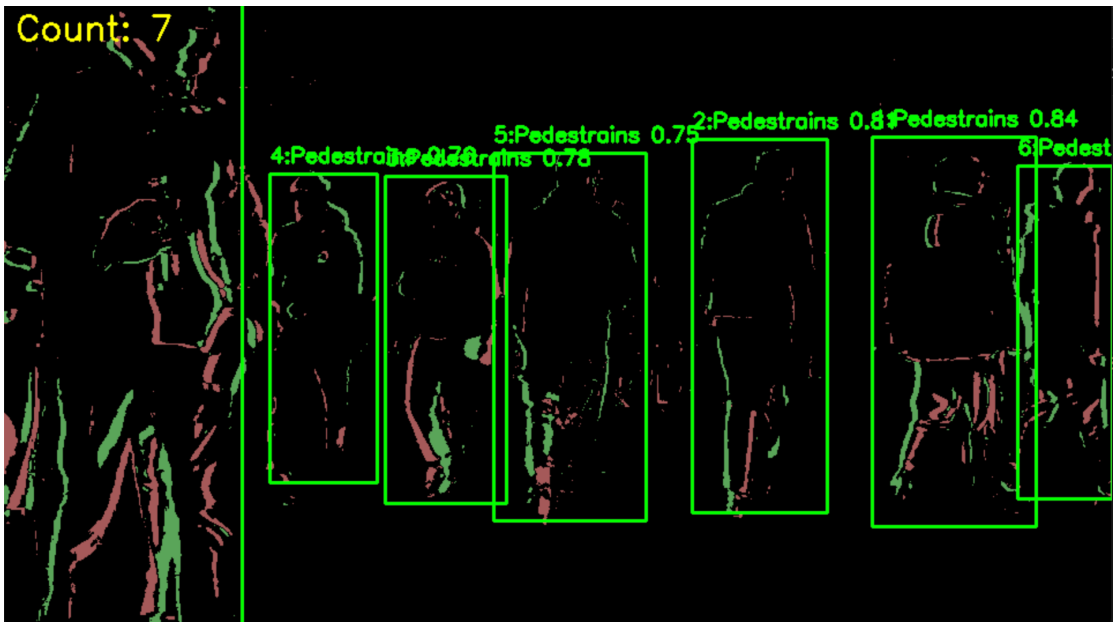
Model	Average FPS (every frame)	Average latency (every frame)	Model size (onnx)	Average Precision (target in each frame)
YOLOv5n	76.92	13.00ms	7.2MB	83.22%
YOLOv5s	60.99	16.40ms	27.1MB	84.33%
YOLOv5m	46.57	21.47ms	80.0MB	85.89%
YOLOv5l	34.22	29.22ms	176.4MB	93.89%
YOLOv5x	23.18	43.14ms	392.3MB	94.78%
YOLOv8n	79.00	12.66ms	11.6MB	81.29%
YOLOv8s	61.20	16.33ms	42.7MB	84.45%
YOLOv8m	53.75	18.60ms	98.8MB	87.67%
YOLOv8l	34.46	29.02ms	166.0MB	94.20%

YOLOv8x	19.37	51.63ms	260.2MB	94.27%

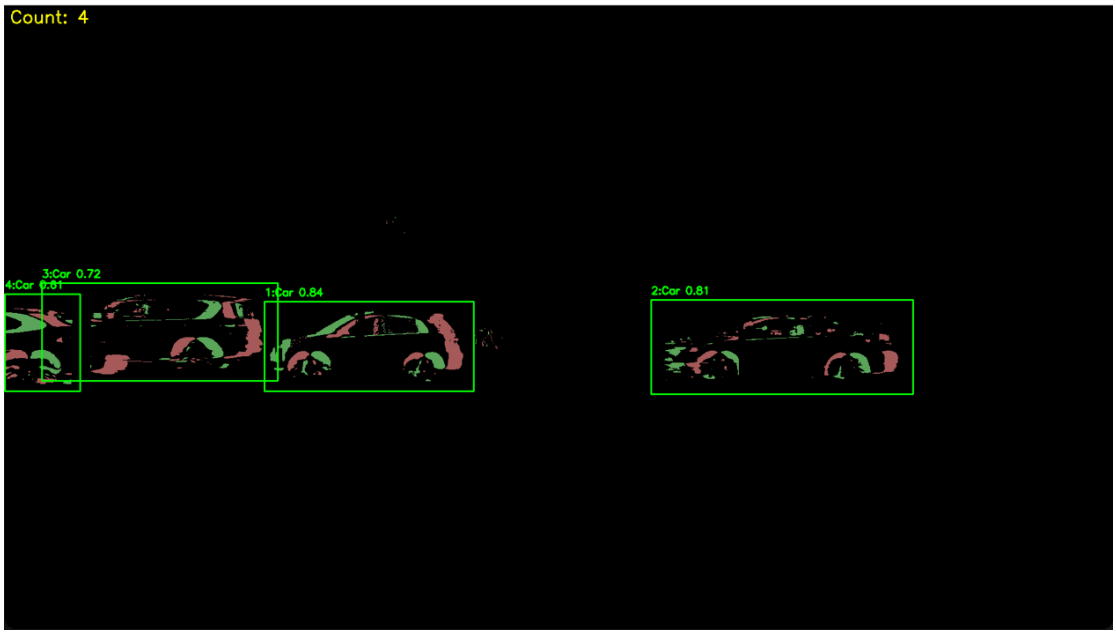
4.5 Counting target objects

The counting function is also applied to the designed monitoring system, which can annotate the number of detected target objects in each frame. It is quite useful to help drivers know whether the road ahead is crowded or spared. The driver can choose to speed up or slow down based on the judgment of road conditions. The deployment results are shown in Fig.39. In Fig.39(a), there are nine riders detected, where the number of targets is labelled above each bounding box and also prints the total number of targets at the top left corner with a yellow line. Applying this function would not affect the detection results and the inference time, it is added only in the main loop. More detection instances are also shown in Fig.39(b) and (c), where the pedestrians and cars are labelled correctly with the correct counting numbers. During the implementation of this designed system, the pedestrians are the most challenging to detect with high precision, because the instances of them are not as obvious as those of riders and cars, increasing the detection difficulty to recognize the contour line for each pedestrian whose dress and walking habits are totally different. However, the detection results under YOLOv5n are still excellent for every trained class in this paper, especially when applying the blending function to increase the RGB rendering for each instance.

(a): Counting the riders in traffic scene based on YOLOv5n



(b): Counting the pedestrians in traffic scene based on YOLOv8n



(c): Counting the cars in the traffic scene based on YOLOv8s

Figure 39: The deployment of the counting function

Chapter 5: Discussion

Based on the results in Chapter 4, compared with the original RGB frame, the event-based frame made an effort to reduce the classification difficulty when there are many abstract objects within a frame. The experiments to simulate DVS with the threshold at 0.8 could capture the moving objects clearly and filter most noises. Fig.22(b) indicated that it was impossible to lower the threshold below 0.5, since there were thousands of unrelated moving actions that could be captured from the backgrounds and noises.

Frequency-based event stream encoding got the best conversion results that kept the consistent moving representations without the changes of intensity for moving objects. Even though the SAE and LIF methods could also convert the event stream into a frame-based image, the phenomenon of motion blurs and changing intensities based on the timestamp on the moving objects would lower the detection accuracy. A large accumulation time caused the significant motion blur, even though finer details could be captured. Fig.24 evidenced that the accumulation time at 0.01 not only minimized the effects of motion blurs and noises but also recovered the most details of moving objects. As shown in Fig.25 and Fig.27, SAE and LIF conducted the poor capturing task when the accumulated more than 0.5ns, while the frequency-based method could tolerate higher accumulation time. The RGB color rendering implemented after preprocessing with a blending function, it was found that the blending output was more suitable to act as the YOLO inputs to train since the contrast level increases to help neurons learn more patterns and features rather than trying to find which features are required to learn in the low illumination contours. Based on the annotated custom dataset, the detection models of YOLOv5 and YOLOv8 were trained under 100 epochs with 16 batch size to compare the validation performances and the trend of training loss curves. The results in Table 3 showed that YOLOv5 achieved more than 90% recall rate for all versions, which might be the most suitable for doing the real-time traffic monitoring task as it minimized the probability of missing the targets. YOLOv8 had the better performance on precision in Table 5, all versions kept more than 90% precision in the validation results. Therefore, there was a trade-off between precision and recall rate. The drop in recall rate could be led by training too many cycles, as the training process showed high precision since it had been over-fitted. The requirement of a high recall rate was more important in this designed monitoring system. The training results in Fig.33 showed that the model was over-fitting when trained with 125 epochs for YOLOv5s, meaning that training of a larger model size requires a smaller number of cycles or use early-stopping to avoid the over-fitting phenomenon. The Training results of YOLOv8n in Fig.35 demonstrated that YOLOv8 could converge more steadily to be able to learn more complicated features, which was quite different from YOLOv5 dropping the loss curve at the earlier epoch. Both YOLOv5 and YOLOv8 were deployed into the main program by implementing them under the TensorFlow Lite and ONNX frameworks. Due to the limitations of Apple M1 silicon chip, the detection performances under M1 CPU in Table 6 were terrible since the smallest YOLOv5n could only be run at 13.22 FPS, which was unacceptable for real-time detection. However, in Tables 7 and 8, the FPS performances had

significant improvements under the CPU of NVIDIA RTX 4060 that could increase FPS to 39.79. What is more, the FPS rises to 79.95 when deploying the ONNX format model of yolov5n to run under the GPU of NVIDIA RTX 4060, which is excellent result to achieve perfect real-time monitoring. The deployment results of YOLOv5 and YOLOv8 in Table 8 indicated that the larger version had better prediction performances, even though the training results showed that YOLOv5n and YOLOv8n had no significant difference for the parameters of precision and recall rate with larger model versions. The counting function was also applied, it aimed to realize the function of the alarm driver when the road ahead was congested. Fig.39 showed that the total counting number could be presented in top left corner, and each target had a labelled number above the bounding box.

Chapter 6: Conclusion

In this paper, an event-based multi-class traffic monitoring system is built based on CNN-based YOLO detectors. The v2e with frequency-based encoding plus blending method is proposed to optimize the implementation performances, achieving the average precision of 83.22% and 81.29% for deploying YOLOv5n and YOLov8n, respectively. Each version's YOLO model is trained to analyze and compare the model performances with the method of variable controlling, exploring the influences of the number of batch size and epochs for the training results. Implementing the model under the CPU and GPU environment with two different hardware to seek the objective detection results and find out the YOLO model feasibility under ONNX and TFLITE frameworks. Realizing the counting function to develop probability and creativity to add more functions to enhance the driver's experiences.

Chapter 7: Future works

Future work 1:

Even though the program is conducted successfully, the phenomenon that training results for smaller models in YOLOv5 and YOLOv8 have better performance than the larger size at the evaluation step is weird. The reason is that the collected custom dataset is still not enough so that the simpler model can perform recall rate and precision nearly or greater than the larger model. In the future, a dataset that contains more than 3000 event streams is required to implement to seek more generalized results. However, in this paper, the current progress has been performed well since the average precision increases as the model size increases during the integration process.

Future work 2:

Despite the counting function has been achieved, more embedded functions still need to be explored. The speaker can activate to output voice by integrating with the output counting numbers. For example, the speaker outputs the voice of ‘Notice! A heavy traffic stream is detected in the surrounding area. Please slow down and drive carefully’ when the monitoring system counts the detection for the number of trained classes of traffic participants above 6 in the current frame. This function can provide more information feedback to driver directly.

Future work 3:

Integrate the designed system into the real hardware is also important, this paper used the simulated neuromorphic camera rather than the real DVS sensor. Then the dataset can be constructed based on the real DVS event stream rather than synthetic data based on v2e. In the future, the program can be deployed with the real edge-computing devices like jetson or Raspberry Pi to develop the complete products with SD card, water-proofing case and ion-lithium battery.

Reference

1. Jain, N.K., Saini, R.K. and Mittal, P., 2018. A review on traffic monitoring system techniques. *Soft computing: Theories and applications: Proceedings of SoCTA 2017*, pp.569-577.
2. Putra, A.S. and Warnars, H.L.H.S., 2018, September. Intelligent traffic monitoring system (ITMS) for smart city based on IoT monitoring. In *2018 Indonesian Association for Pattern Recognition International Conference (INAPR)* (pp. 161-165). IEEE.
3. Kadkhodayi, A., Jabeli, M., Aghdam, H. and Mirbakhsh, S., 2023. Artificial Intelligence-Based Real-Time Traffic Man-agement. *Journal of Electrical Electronics Engineering*, *2*(4), pp.368-73.
4. Theuwissen, A., 2014. CMOS image sensors. *Smart Sensor Systems: Emerging Technologies and Applications*, pp.173-189.
5. Rebecq, H., Ranftl, R., Koltun, V. and Scaramuzza, D., 2019. High speed and high dynamic range video with an event camera. *IEEE transactions on pattern analysis and machine intelligence*, *43*(6), pp.1964-1980.
6. Jiang, Z., Zhao, L., Li, S. and Jia, Y., 2020. Real-time object detection method based on improved YOLOv4-tiny. *arXiv preprint arXiv:2011.04244*.
7. Dalal, N. and Triggs, B., 2005, June. Histograms of oriented gradients for human detection. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)* (Vol. 1, pp. 886-893). Ieee.
8. Ward, I.R., Laga, H. and Bennamoun, M., 2019. RGB-D image-based object detection: from traditional methods to deep learning techniques. *RGB-D Image Analysis and Processing*, pp.169-201.
9. Zhao, Z.Q., Zheng, P., Xu, S.T. and Wu, X., 2019. Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, *30*(11), pp.3212-3232.
10. Zhiqiang, W. and Jun, L., 2017, July. A review of object detection based on convolutional neural network. In *2017 36th Chinese control conference (CCC)* (pp. 11104-11109). IEEE.
11. Shenai, H., Gala, J., Kekre, K., Chitale, P. and Karani, R., 2022. Combating COVID-19 using object detection techniques for next-generation autonomous systems. *Cyber-Physical Systems. Academic Press*, pp. 55–73.
12. M. Maity, S. Banerjee and S. Sinha Chaudhuri, "Faster R-CNN and YOLO based Vehicle detection: A Survey," *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, Erode, India, 2021, pp. 1442-1447.
13. Sumit, S.S., Watada, J., Roy, A. and Rambli, D.R.A., 2020, April. In object detection deep learning methods, YOLO shows supremum to Mask R-CNN. In *Journal of Physics: Conference Series* (Vol. 1529, No. 4, p. 042086). IOP Publishing.

14. Ćorović, A., Ilić, V., Đurić, S., Marijan, M. and Pavković, B., 2018, November. The real-time detection of traffic participants using YOLO algorithm. In *2018 26th Telecommunications Forum (TELFOR)* (pp. 1-4). IEEE.
15. Chakravarthi, B., Verma, A.A., Daniilidis, K., Fermuller, C. and Yang, Y., 2025. Recent event camera innovations: A survey. In *European Conference on Computer Vision* (pp. 342-376). Springer, Cham.
16. Liu, Z., Chen, G., Wu, Y., Du, J., Conradt, J. and Knoll, A., 2022. Mixed Event-Frame Vision System for Daytime Preceding Vehicle Taillight Signal Measurement Using Event-Based Neuromorphic Vision Sensor. *Journal of Advanced Transportation*, *2022*(1), p.2673191.
17. Liao, F., Zhou, F. and Chai, Y., 2021. Neuromorphic vision sensors: Principle, progress and perspectives. *Journal of Semiconductors*, *42*(1), p.013105.
18. Wan, J., Xia, M., Huang, Z., Tian, L., Zheng, X., Chang, V., Zhu, Y. and Wang, H., 2021. Event-based pedestrian detection using dynamic vision sensors. *Electronics*, *10*(8), p.888.
19. Chen, N.F., 2018. Pseudo-labels for supervised learning on dynamic vision sensor data, applied to object detection under ego-motion. *In Proceedings of the IEEE conference on computer vision and pattern recognition workshops* (pp. 644-653).
20. Dewangan, D.K. and Sahu, S.P., 2020. Driving behavior analysis of intelligent vehicle system for lane detection using vision-sensor. *IEEE Sensors Journal*, 21(5), (pp. 6367-6375).
21. Chen, X., 2023. Autonomous Driving using Spiking Neural Networks on Dynamic Vision Sensor Data: A Case Study of Traffic Light Change Detection. arXiv preprint arXiv:2311.09225.
22. Su, Q., Chou, Y., Hu, Y., Li, J., Mei, S., Zhang, Z. and Li, G., 2023. Deep directly-trained spiking neural networks for object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 6555-6565).
23. Glučina, M., Anđelić, N., Lorencin, I. and Car, Z., 2023. Detection and classification of printed circuit boards using YOLO algorithm. *Electronics*, *12*(3), p.667.
24. Jegham, N., Koh, C.Y., Abdelatti, M. and Hendawi, A., 2024. Evaluating the evolution of yolo (you only look once) models: A comprehensive benchmark study of yolo11 and its predecessors. *arXiv preprint arXiv:2411.00201*.
25. Chakravarthi, B., Verma, A.A., Daniilidis, K., Fermuller, C. and Yang, Y., 2024, September. Recent event camera innovations: A survey. In *European Conference on Computer Vision* (pp. 342-376). Cham: Springer Nature Switzerland.
26. Han, H., Lyu, J., Li, J., Wei, H., Li, C., Wei, Y., Chen, S. and Ji, X., 2024, September. Physical-Based Event Camera Simulator. In *European Conference on Computer Vision* (pp. 19-35). Cham: Springer Nature Switzerland.

27. Chen, G., Cao, H., Ye, C., Zhang, Z., Liu, X., Mo, X., Qu, Z., Conradt, J., Röhrbein, F. and Knoll, A., 2019. Multi-cue event information fusion for pedestrian detection with neuromorphic vision sensors. *Frontiers in neurorobotics*, *13*, p.10.
28. Gallego, G., Delbrück, T., Orchard, G., Bartolozzi, C., Taba, B., Censi, A., Leutenegger, S., Davison, A.J., Conradt, J., Daniilidis, K. and Scaramuzza, D., 2020. Event-based vision: A survey. *IEEE transactions on pattern analysis and machine intelligence*, *44*(1), pp.154-180.
29. Lu, D., Kong, L., Lee, G.H., Chane, C.S. and Ooi, W.T., 2024. FlexEvent: Event Camera Object Detection at Arbitrary Frequencies. *arXiv preprint arXiv:2412.06708*.
30. Bai, D., Wei, S., He, X. and Yu, Q., 2025, March. Annotation Methods for Object Detection: A Comparative Analysis from Manual Labeling to Automated Annotation Technologies. In *2025 5th International Conference on Artificial Intelligence and Industrial Technology Applications (AIITA)* (pp. 1473-1479). IEEE.
31. Jiang, T., Frøseth, G.T. and Rønnquist, A., 2023. A robust bridge rivet identification method using deep learning and computer vision. *Engineering Structures*, *283*, p.115809.
32. Jocher, G., Stoken, A., Borovec, J., Changyu, L., Hogan, A., Diaconu, L., Poznanski, J., Yu, L., Rai, P., Ferriday, R. and Sullivan, T., 2020. ultralytics/yolov5: v3. 0. *Zenodo*.
33. Sohan, M., Sai Ram, T. and Rami Reddy, C.V., 2024. A review on yolov8 and its advancements. In *International Conference on Data Intelligence and Cognitive Informatics* (pp. 529-545). Springer, Singapore.
34. Jabbar, H. and Khan, R.Z., 2015. Methods to avoid over-fitting and under-fitting in supervised machine learning (comparative study). *Computer science, communication and instrumentation devices*, *70*(10.3850), pp.978-981.
35. Ali, U., Ismail, M.A., Habeeb, R.A.A. and Shah, S.R.A., 2024. Performance evaluation of YOLO models in plant disease detection. *Journal of Informatics and Web Engineering*, *3*(2), pp.199-211.

Appendix

A: The Python code for the deployment overall system under CPU

```
import cv2
import time
import os
import numpy as np
import tensorflow as tf
from collections import deque
from tensorflow.lite.python.interpreter import Interpreter

# === Paths & settings ===
VIDEO_PATH  = "/Users/huanghaiyang/Desktop/HKU_FYP/3691361-
hd_1920_1080_30fps.mp4"
TFLITE_PATH = "/Users/huanghaiyang/Desktop/yolov5n.tflite"
YOLO_THRESHOLD   = 0.7
INPUT_SIZE      = 640
NUM_THREADS     = 8   # for TFLite CPU

# DVS encoding params
DVS_THRESHOLD      = 0.8
ACCUMULATION_TIME   = 0.001
RESIZE_SCALE       = 0.5
EVENT_GAIN         = 50
ALPHA              = 0.98

# === Open video & compute minimal delay ===
cap = cv2.VideoCapture(VIDEO_PATH)
raw_fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
delay_ms = 1
print(f"🔢 Video FPS={raw_fps:.2f}, delay={delay_ms}ms")

# === Class maps ===
class_idx2name = {0:"Car", 1:"Rider", 2:"Pedestrains"}
```

```
# === Letterbox helper ===
def letterbox(img, new_shape=(INPUT_SIZE,INPUT_SIZE), color=(114,114,114)):
    h0, w0 = img.shape[:2]
    r = min(new_shape[0]/h0, new_shape[1]/w0)
    nh, nw = int(round(h0*r)), int(round(w0*r))
    dh, dw = new_shape[0]-nh, new_shape[1]-nw
    dh, dw = dh/2, dw/2
    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_LINEAR)
    top, bottom = int(round(dh-0.1)), int(round(dh+0.1))
    left, right = int(round(dw-0.1)), int(round(dw+0.1))
    padded = cv2.copyMakeBorder(resized, top, bottom, left, right,
                                cv2.BORDER_CONSTANT, value=color)
    return padded, r, (dw, dh)

# === TFLite setup ===
interpreter = Interpreter(model_path=TFLITE_PATH, num_threads=NUM_THREADS)
interpreter.allocate_tensors()
i_idx = interpreter.get_input_details()[0]['index']
o_idx = interpreter.get_output_details()[0]['index']

# === Initialize DVS state ===
ret, frame0 = cap.read()
if not ret:
    raise RuntimeError("Cannot read first frame")
if RESIZE_SCALE != 1.0:
    frame0 = cv2.resize(frame0, (0,0), fx=RESIZE_SCALE, fy=RESIZE_SCALE)
prev_gray = cv2.cvtColor(frame0, cv2.COLOR_BGR2GRAY).astype(np.float32)
prev_gray = np.log1p(prev_gray + 1)
h, w = prev_gray.shape
event_counter = np.zeros((h,w), dtype=np.int32)
event_buffer  = deque()

# === YOLO inference on BGR image ===
def run_yolo(img_bgr):
    img_p, r, (dw,dh) = letterbox(img_bgr)
    inp = np.expand_dims(img_p.astype(np.float32)/255.0, 0)
```

```
    interpreter.set_tensor(i_idx, inp)
    interpreter.invoke()
    output = interpreter.get_tensor(o_idx)[0]
    boxes, scores, classes = [], [], []
    for det in output:
        conf = float(det[4])
        if conf < YOLO_THRESHOLD: continue
        x,y,bw,bh = det[:4]*INPUT_SIZE
        x = (x-dw)/r; y = (y-dh)/r
        bw/=r; bh/=r
        x1,y1 = int(x-bw/2), int(y-bh/2)
        x2,y2 = int(x+bw/2), int(y+bh/2)
        boxes.append((x1,y1,x2,y2))
        scores.append(conf)
        classes.append(int(np.argmax(det[5:])))
    dets=[]
    if boxes:
        keep = tf.image.non_max_suppression(
            boxes, scores,
            max_output_size=len(boxes),
            iou_threshold=0.5,
            score_threshold=YOLO_THRESHOLD
        ).numpy()
        for i in keep:
            dets.append((boxes[i], scores[i], classes[i]))
    return dets

# === Main loop: DVS encode → YOLO → display YOLO ===
frame_idx = 0
fps_buf = deque(maxlen=30)
t_start = time.time()

while True:
    t0 = time.time()
    ret, frame = cap.read()
    if not ret: break
```

```
    if RESIZE_SCALE!=1.0:
        frame = cv2.resize(frame,(0,0),fx=RESIZE_SCALE,fy=RESIZE_SCALE)

    # DVS encode
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
    gray = np.log1p(gray + 1)
    delta = gray - prev_gray
    prev_gray[:] = gray

    pos = (delta > DVS_THRESHOLD)
    neg = (delta < -DVS_THRESHOLD)

    now = time.time()
    event_counter[pos] += 1
    event_counter[neg] += 1
    event_buffer.append((now, pos.copy(), neg.copy()))
    while event_buffer and now - event_buffer[0][0] > ACCUMULATION_TIME:
        _, p_old, n_old = event_buffer.popleft()
        event_counter[p_old] -= 1
        event_counter[n_old] -= 1

    cnt = event_counter.copy()
    cnt[np.abs(cnt)<1] = 0
    sc = cnt - 1
    sigmoid = 255.0/(1+np.exp(-sc/2))
    freq = np.where(cnt==0, 0, sigmoid).astype(np.uint8)

    evt_rgb = np.zeros((h,w,3), np.uint8)
    evt_rgb[pos] = (0,255,0)
    evt_rgb[neg] = (0,0,255)

    freq_bgr = cv2.cvtColor(freq, cv2.COLOR_GRAY2BGR)
    blended  = cv2.addWeighted(freq_bgr,0.7,evt_rgb,0.3,0)

    # YOLO on blended DVS
    dets = run_yolo(blended)
```

```
    for (x1,y1,x2,y2), conf, cls in dets:
        lbl = class_idx2name[cls]
        cv2.rectangle(blended, (x1,y1),(x2,y2), (0,255,0),2)
        cv2.putText(blended,f"{lbl} {conf:.2f}",(x1,y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,0.5,(0,255,0),2)

    # Display only blended + YOLO
    cv2.imshow("DVS + YOLOv5", blended)

    # FPS logging
    fps_buf.append(time.time()-t0)
    if frame_idx%10==0 and len(fps_buf)>1:
        fps = 1.0/(sum(fps_buf)/len(fps_buf))
        print(f"[{frame_idx}] ≈ {fps:.2f} FPS")

    frame_idx += 1
    if cv2.waitKey(delay_ms)&0xFF==ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
print(f"Finished {frame_idx} frames in {time.time()-t_start:.2f}s → "
      f"{frame_idx/(time.time()-t_start):.2f} FPS")
```

B: The Python code for the deployment overall system under GPU

```
import cv2
import time
import os
import numpy as np
import tensorflow as tf
import onnxruntime as ort
from collections import deque

# === Paths & settings ===
```

```
VIDEO_PATH     = "/Users/huanghaiyang/Desktop/HKU_FYP/3691361-
hd_1920_1080_30fps.mp4"
ONNX_PATH      = "/Users/huanghaiyang/Desktop/yolov5s.onnx"    # ONNX model
here
YOLO_THRESHOLD  = 0.6
INPUT_SIZE     = 640
# DVS encoding params
DVS_THRESHOLD    = 0.8
ACCUMULATION_TIME = 0.001
SAVE_INTERVAL    = 30
RESIZE_SCALE     = 0.5
# Blending / display
EVENT_GAIN       = 50
ALPHA           = 0.98

OUTPUT_DIR = "dvs_frames"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# === Video & delay setup ===
cap    = cv2.VideoCapture(VIDEO_PATH)
raw_fps= cap.get(cv2.CAP_PROP_FPS) or 30.0
delay_ms = 1
print(f"🔢 Video FPS={raw_fps:.2f}, display delay={delay_ms}ms")

# === Letterbox helper ===
def letterbox(img, new_shape=(INPUT_SIZE,INPUT_SIZE), color=(114,114,114)):
    h0, w0 = img.shape[:2]
    r     = min(new_shape[0]/h0, new_shape[1]/w0)
    nh, nw = int(round(h0*r)), int(round(w0*r))
    dh, dw = (new_shape[0]-nh)/2, (new_shape[1]-nw)/2

    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_LINEAR)
    top, bottom = int(round(dh-0.1)), int(round(dh+0.1))
    left, right = int(round(dw-0.1)), int(round(dw+0.1))
    padded = cv2.copyMakeBorder(resized, top, bottom, left, right,
                                cv2.BORDER_CONSTANT, value=color)
```

```
    return padded, r, (dw, dh)

# === ONNX Runtime session (GPU preferred) ===
sess_opts = ort.SessionOptions()
probe    = ort.InferenceSession(ONNX_PATH, sess_opts,
providers=ort.get_available_providers())
avail    = probe.get_providers()
for p in
("CUDAExecutionProvider","DmlExecutionProvider","CPUExecutionProvider"):
    if p in avail:
        providers = [p]
        break
print("Using ONNX provider:", providers)
session   = ort.InferenceSession(ONNX_PATH, sess_opts, providers=providers)
input_name= session.get_inputs()[0].name

# === Class maps ===
class_idx2name = {0:"Car", 1:"Rider", 2:"Pedestrains"}

# === Initialize DVS state ===
ret, frame0 = cap.read()
if not ret:
    raise RuntimeError("Cannot read first frame")
if RESIZE_SCALE != 1.0:
    frame0 = cv2.resize(frame0, (0,0), fx=RESIZE_SCALE, fy=RESIZE_SCALE)
prev_gray = cv2.cvtColor(frame0, cv2.COLOR_BGR2GRAY).astype(np.float32)
prev_gray = np.log1p(prev_gray + 1)
h, w = prev_gray.shape
event_counter = np.zeros((h,w), dtype=np.int32)
event_buffer  = deque()

# === ONNX inference on blended BGR image ===
def run_onnx_yolo(img_bgr):
    img_p, r, (dw,dh) = letterbox(img_bgr)
    blob = img_p.astype(np.float32) / 255.0
    blob = np.transpose(blob, (2,0,1))[None,...]  # (1,3,640,640)
```

```
raw = session.run(None, {input_name: blob})[0]
preds = raw[0] if raw.ndim==3 else raw       # shape: (N,5+classes)

boxes, scores, classes = [], [], []
for det in preds:
    obj_c = float(det[4])
    if obj_c < 1e-3:
        continue
    cls_probs = det[5:]
    cls      = int(np.argmax(cls_probs))
    cls_c    = float(cls_probs[cls])
    score    = obj_c * cls_c
    if score < YOLO_THRESHOLD:
        continue

    x,y,bw,bh = det[:4] * INPUT_SIZE
    x = (x - dw)/r; y = (y - dh)/r
    bw/= r; bh/= r

    x1 = int(x - bw/2); y1 = int(y - bh/2)
    x2 = int(x + bw/2); y2 = int(y + bh/2)

    boxes.append((x1,y1,x2,y2))
    scores.append(score)
    classes.append(cls)

dets = []
if boxes:
    keep = tf.image.non_max_suppression(
        boxes, scores,
        max_output_size=len(boxes),
        iou_threshold=0.5,
        score_threshold=YOLO_THRESHOLD
    ).numpy()
    for i in keep:
```

```
            dets.append((boxes[i], scores[i], classes[i]))
    return dets

# === Main loop: DVS encode → blend → ONNX YOLO → display ===
frame_idx = 0
t_start   = time.time()

while True:
    t0 = time.time()
    ret, frame = cap.read()
    if not ret:
        break
    if RESIZE_SCALE != 1.0:
        frame = cv2.resize(frame, (0,0), fx=RESIZE_SCALE, fy=RESIZE_SCALE)

    # — DVS encoding —
    gray  = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
    log_i = np.log1p(gray + 1)
    delta = log_i - prev_gray
    prev_gray[:] = log_i

    pos = (delta > DVS_THRESHOLD)
    neg = (delta < -DVS_THRESHOLD)

    now = time.time()
    event_counter[pos] += 1
    event_counter[neg] += 1
    event_buffer.append((now, pos.copy(), neg.copy()))
    while event_buffer and now - event_buffer[0][0] > ACCUMULATION_TIME:
        _, p_old, n_old = event_buffer.popleft()
        event_counter[p_old] -= 1
        event_counter[n_old] -= 1

    cnt = event_counter.copy()
    cnt[np.abs(cnt)<1] = 0
    sc  = cnt - 1
```

```
    sigmoid = 255.0/(1 + np.exp(-sc/2))
    freq = np.where(cnt==0, 0, sigmoid).astype(np.uint8)

    evt_rgb  = np.zeros((h,w,3), dtype=np.uint8)
    evt_rgb[pos] = (0,255,0)
    evt_rgb[neg] = (0,0,255)

    freq_bgr = cv2.cvtColor(freq, cv2.COLOR_GRAY2BGR)
    blended  = cv2.addWeighted(freq_bgr, 0.7, evt_rgb, 0.3, 0)

    # — Save DVS frames occasionally —
    if frame_idx % SAVE_INTERVAL == 0:
        cv2.imwrite(os.path.join(OUTPUT_DIR, f"dvs_{frame_idx:05d}.png"),
blended)

    # — ONNX YOLO on blended image —
    dets = run_onnx_yolo(blended)

    # annotate & draw
    cv2.putText(blended, f"Count: {len(dets)}", (10,30),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0,255,255), 2)
    for idx, ((x1,y1,x2,y2), conf, cls) in enumerate(dets, start=1):
        label = class_idx2name.get(cls, "unknown")
        text  = f"{idx}:{label} {conf:.2f}"
        cv2.rectangle(blended, (x1,y1), (x2,y2), (0,255,0), 2)
        cv2.putText(blended, text, (x1, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)

    cv2.imshow("DVS + YOLOv5 (ONNX)", blended)

    # FPS log
    if frame_idx % 10 == 0:
        fps = 1.0/((time.time()-t0) or 1e-6)
        print(f"[{frame_idx}] ~{fps:.2f} FPS")

    frame_idx += 1
```

```
    if cv2.waitKey(delay_ms) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
print(f"Processed {frame_idx} frames in {time.time()-t_start:.2f}s "
      f"→ {frame_idx/(time.time()-t_start):.2f} FPS")
```

C: DVS simulation plus frequency-based encoding

```
import cv2
import numpy as np
import time
import os
from collections import deque

# Parameters
threshold = 0.8
accumulation_time = 0.001  # short window for sharper object response
fps = 30
save_interval = 1
resize_scale = 0.5
show_gui = True

output_dir = "dvs_frames"
os.makedirs(output_dir, exist_ok=True)

# Load video
cap =
cv2.VideoCapture("/Users/huanghaiyang/Desktop/HKU_FYP/12603631_3840_2160_30fp
s.mp4")
ret, prev_frame = cap.read()
if not ret:
    print("Error: Could not read the video file.")
    cap.release()
```

```
    exit()

# Resize for speed
if resize_scale != 1.0:
    prev_frame = cv2.resize(prev_frame, (0, 0), fx=resize_scale,
fy=resize_scale)

prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
prev_gray = np.log1p(prev_gray + 1)

height, width = prev_gray.shape
event_counter = np.zeros((height, width), dtype=np.int32)
event_buffer = deque()

frame_idx = 0
fps_times = deque(maxlen=30)

while True:
    t0 = time.time()
    ret, frame = cap.read()
    if not ret:
        break

    if resize_scale != 1.0:
        frame = cv2.resize(frame, (0, 0), fx=resize_scale, fy=resize_scale)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
    gray = np.log1p(gray + 1)

    delta = gray - prev_gray
    events_pos = (delta > threshold)
    events_neg = (delta < -threshold)

    event_counter += events_pos.astype(np.int32)
    event_counter += events_neg.astype(np.int32)
```

```
    now = time.time()
    event_buffer.append((now, events_pos, events_neg))

    while event_buffer and now - event_buffer[0][0] > accumulation_time:
        _, old_pos, old_neg = event_buffer.popleft()
        event_counter -= old_pos.astype(np.int32)
        event_counter -= old_neg.astype(np.int32)

    # === Noise filtering and sigmoid mapping ===
    filtered_counter = event_counter.copy()
    filtered_counter[np.abs(filtered_counter) < 1] = 0  # remove weak/noisy
pixels
    shifted_counter = filtered_counter - 1            # shift 0-event baseline
toward darker

    # Apply sigmoid normalization
    sigmoid_frame = 255 / (1 + np.exp(-shifted_counter.astype(np.float32) /
2))
    sigmoid_frame = sigmoid_frame.astype(np.uint8)

    # Final black-background frequency frame ===
    freq_frame = np.where(filtered_counter == 0, 0,
sigmoid_frame).astype(np.uint8)

    # Save output image
    if frame_idx % save_interval == 0:
        out_path = os.path.join(output_dir,
f"dvs_freq_frame_{frame_idx:05d}.png")
        cv2.imwrite(out_path, freq_frame)

    # Display
    if show_gui:
        cv2.imshow("DVS Frequency-Based Frame", freq_frame)

        # Optional: show instantaneous polarity events
        event_frame = np.zeros_like(gray, dtype=np.uint8)
```

```
        event_frame[events_pos] = 255
        event_frame[events_neg] = 127
        cv2.imshow("DVS Instantaneous Events", event_frame)

    # FPS monitor
    t1 = time.time()
    fps_times.append(t1 - t0)
    if frame_idx % 10 == 0 and len(fps_times) >= 2:
        avg_fps = 1 / (sum(fps_times) / len(fps_times))
        print(f"[Frame {frame_idx}] Approx FPS: {avg_fps:.2f}")

    prev_gray = gray
    frame_idx += 1

    if show_gui and cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

D: DVS simulation plus SAE-based encoding

```
import cv2
import numpy as np
import time
import os

# === Parameters ===
threshold = 0.8
tau = 0.5   # Decay time constant for SAE (seconds)
fps = 30
resize_scale = 0.5
show_gui = True
save_interval = 1

output_dir = "dvs_sae_frames"
```

```
os.makedirs(output_dir, exist_ok=True)

# === Load video ===
cap = cv2.VideoCapture("/Users/huanghaiyang/Desktop/HKU_FYP/4779862-
hd_1920_1080_30fps.mp4")
ret, prev_frame = cap.read()
if not ret:
    print("Error: Could not read video.")
    cap.release()
    exit()

if resize_scale != 1.0:
    prev_frame = cv2.resize(prev_frame, (0,0), fx=resize_scale,
fy=resize_scale)

prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
prev_gray = np.log1p(prev_gray + 1)

height, width = prev_gray.shape
# === Initialize SAE with zeros ===
sae = np.zeros((height, width), dtype=np.float32)

frame_idx = 0
fps_times = []

start_time = time.time()

while True:
    t0 = time.time()
    ret, frame = cap.read()
    if not ret:
        break

    if resize_scale != 1.0:
        frame = cv2.resize(frame, (0,0), fx=resize_scale, fy=resize_scale)
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
    gray = np.log1p(gray + 1)

    delta = gray - prev_gray
    events_pos = (delta > threshold)
    events_neg = (delta < -threshold)

    # === Update SAE with current timestamps ===
    timestamp = time.time() - start_time  # relative timestamp

    sae[events_pos] = timestamp
    sae[events_neg] = timestamp

    # === Compute decay map ===
    time_since_last = timestamp - sae
    sae_frame = np.exp(-time_since_last / tau)
    sae_frame = (sae_frame * 255).clip(0,255).astype(np.uint8)

    # Save frame
    if frame_idx % save_interval == 0:
        out_path = os.path.join(output_dir, f"sae_frame_{frame_idx:05d}.png")
        cv2.imwrite(out_path, sae_frame)

    # Show SAE frame
    if show_gui:
        cv2.imshow("DVS SAE", sae_frame)

        # Optional: show raw instantaneous events
        event_frame = np.zeros_like(gray, dtype=np.uint8)
        event_frame[events_pos] = 255
        event_frame[events_neg] = 127
        cv2.imshow("DVS Instantaneous Events", event_frame)

    # FPS
    t1 = time.time()
    fps_times.append(t1 - t0)
```

```
    if len(fps_times) > 30:
        fps_times.pop(0)
    if frame_idx % 10 == 0 and fps_times:
        avg_fps = 1 / (sum(fps_times)/len(fps_times))
        print(f"[Frame {frame_idx}] Approx FPS: {avg_fps:.2f}")

    prev_gray = gray
    frame_idx += 1

    if show_gui and cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
print(f"Finished SAE frames saved to: '{output_dir}'")
```

E: DVS simulation plus LIF-based encoding

```
import cv2
import numpy as np
import os

# === PARAMETERS ===
VIDEO_PATH = "/Users/huanghaiyang/Desktop/HKU_FYP/4779862-
hd_1920_1080_30fps.mp4"
OUTPUT_DIR = "output_lif_frames"
THRESHOLD = 0.8             # DVS threshold in log domain
DECAY_FACTOR = 0.01          # Leak rate per frame (0~1)
EPSILON = 0.05              # Minimum potential to zero out noise
RESIZE_SCALE = 0.5           # Resize for speed
SAVE_INTERVAL = 1            # Save every frame
SHOW_GUI = True             # Display windows

# === SETUP ===
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
cap = cv2.VideoCapture(VIDEO_PATH)
ret, prev_frame = cap.read()
if not ret:
    print("❌ Could not read the video.")
    cap.release()
    exit()

if RESIZE_SCALE != 1.0:
    prev_frame = cv2.resize(prev_frame, (0, 0), fx=RESIZE_SCALE,
fy=RESIZE_SCALE)

prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
prev_gray = np.log1p(prev_gray + 1)

height, width = prev_gray.shape
membrane_potential = np.zeros((height, width), dtype=np.float32)

frame_idx = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    if RESIZE_SCALE != 1.0:
        frame = cv2.resize(frame, (0, 0), fx=RESIZE_SCALE, fy=RESIZE_SCALE)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.float32)
    gray = np.log1p(gray + 1)

    delta = gray - prev_gray
    events_pos = delta > THRESHOLD
    events_neg = delta < -THRESHOLD

    # === LIF INTEGRATION ===
```

```
    membrane_potential += events_pos.astype(np.float32)
    membrane_potential -= events_neg.astype(np.float32)

    # === LEAK ===
    membrane_potential *= DECAY_FACTOR

    # === CLAMP SMALL VALUES ===
    membrane_potential[np.abs(membrane_potential) < EPSILON] = 0

    # === CLIP TO RANGE (optional) ===
    membrane_potential = np.clip(membrane_potential, -10, 10)

    # === VISUALIZATION ===
    # Option 1: highlight ON events only
    mapped = np.clip(membrane_potential, 0, None)
    if np.max(mapped) > 0:
        lif_frame = (mapped / np.max(mapped)) * 255.0
    else:
        lif_frame = mapped
    lif_frame = lif_frame.astype(np.uint8)

    # === SAVE ===
    if frame_idx % SAVE_INTERVAL == 0:
        out_path = os.path.join(OUTPUT_DIR, f"lif_frame_{frame_idx:05d}.png")
        cv2.imwrite(out_path, lif_frame)

    # === DISPLAY ===
    if SHOW_GUI:
        cv2.imshow("LIF DVS", lif_frame)

        # Also show instantaneous polarity for debugging
        event_frame = np.zeros_like(gray, dtype=np.uint8)
        event_frame[events_pos] = 255
        event_frame[events_neg] = 127
        cv2.imshow("Instant Polarity Events", event_frame)
```

```
    # === NEXT ===
    prev_gray = gray.copy()
    frame_idx += 1

    if SHOW_GUI and cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

print(f"Done! Exported LIF frames to '{OUTPUT_DIR}'")
```